

UNIT – 5

SORTING AND SEARCHING

CONTENT

- Basics about Sorting
- BUBBLE SORT
- INSERTION SORT
- SELECTION SORT
- MERGE SORT
- QUICK SORT
- SEQUENTIAL SEARCH
- BINARY SEARCH

Basics

- **Input:**

- A sequence of n numbers $a_1, a_2, \dots, a_n$

- **Output:**

- A permutation (reordering) $a_1', a_2', \dots, a_n'$ of the input sequence such that $a_1' \leq a_2' \leq \dots \leq a_n'$

Some Sorting Definition

- Internal Sort
 - The data to be sorted is all stored in the computer's main memory [Primary memory].
- External Sort
 - Some of the data to be sorted might be stored in some external, slower, device.
- In Place Sort
 - The amount of extra space required to sort the data is constant with the input size.
- Comparison sort:
 - Lower bound: $\Omega(n \lg n)$.
- Non comparison sort:
 - Bucket sort, counting sort, radix sort
 - They are possible in linear time (under certain assumption).

Stability

- A **STABLE** sort preserves relative order of records with equal keys

Sorted on first key:

Aaron	4	A	664-480-0023	097 Little
Andrews	3	A	874-088-1212	121 Whitman
Battle	4	C	991-878-4944	308 Blair
Chen	2	A	884-232-5341	11 Dickinson
Fox	1	A	243-456-9091	101 Brown
Furia	3	A	766-093-9873	22 Brown
Gazsi	4	B	665-303-0266	113 Walker
Kanaga	3	B	898-122-9643	343 Forbes
Rohde	3	A	232-343-5555	115 Holder
Quilici	1	C	343-987-5642	32 McCosh

Sort file on second key:

Records with key value 3
are not in order on first
key!!

Fox	1	A	243-456-9091	101 Brown
Quilici	1	C	343-987-5642	32 McCosh
Chen	2	A	884-232-5341	11 Dickinson
Kanaga	3	B	898-122-9643	343 Forbes
Andrews	3	A	874-088-1212	121 Whitman
Furia	3	A	766-093-9873	22 Brown
Rohde	3	A	232-343-5555	115 Holder
Battle	4	C	991-878-4944	308 Blair
Gazsi	4	B	665-303-0266	113 Walker
Aaron	4	A	664-480-0023	097 Little

Bubble Sort

Alg.: BUBBLESORT(A)

for $i \leftarrow 0$ **to** $\text{length}[A]$

for $j \leftarrow i+1$ **to** $\text{length}[A]$

if $A[i] > A[j]$

then exchange $A[i] \leftrightarrow A[j]$

Insertion Sort

Alg.: INSERTION-SORT(A)

for $j \leftarrow 2$ **to** n

do $\text{key} \leftarrow A[j]$

 // Insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$

$i = j - 1$

while $i > 0$ and $A[i] > \text{key}$

do $A[i + 1] = A[i]$

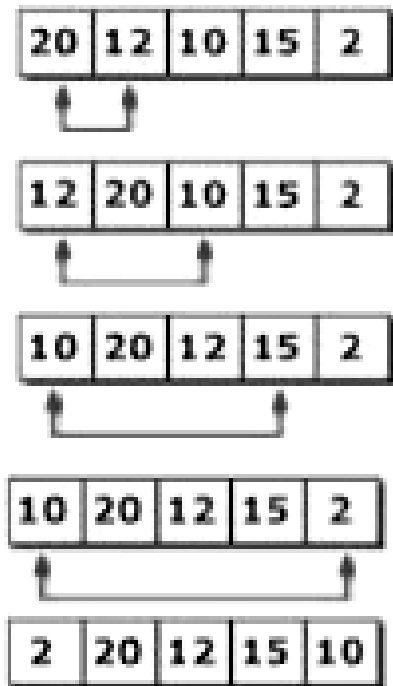
$i = i - 1$

End while

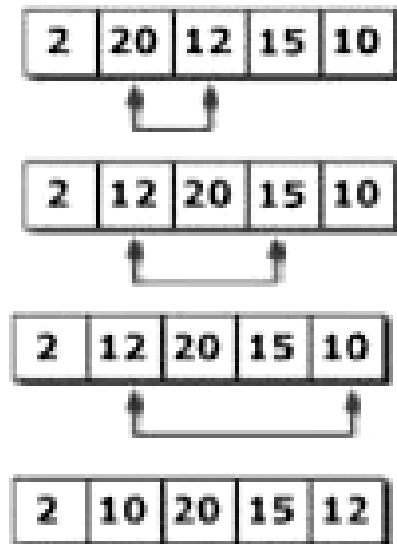
$A[i + 1] = \text{key}$

- Insertion sort – sorts the elements in place

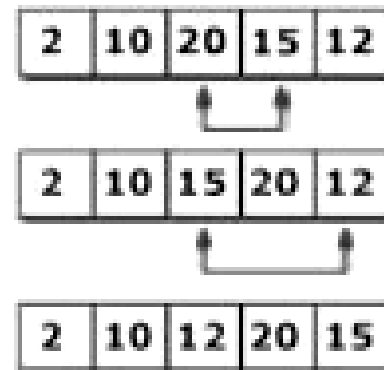
Bubble Sort



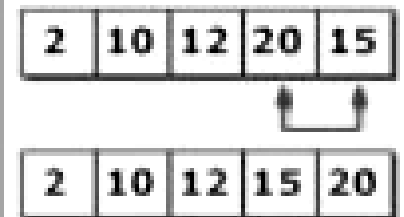
Step 1



Step 2

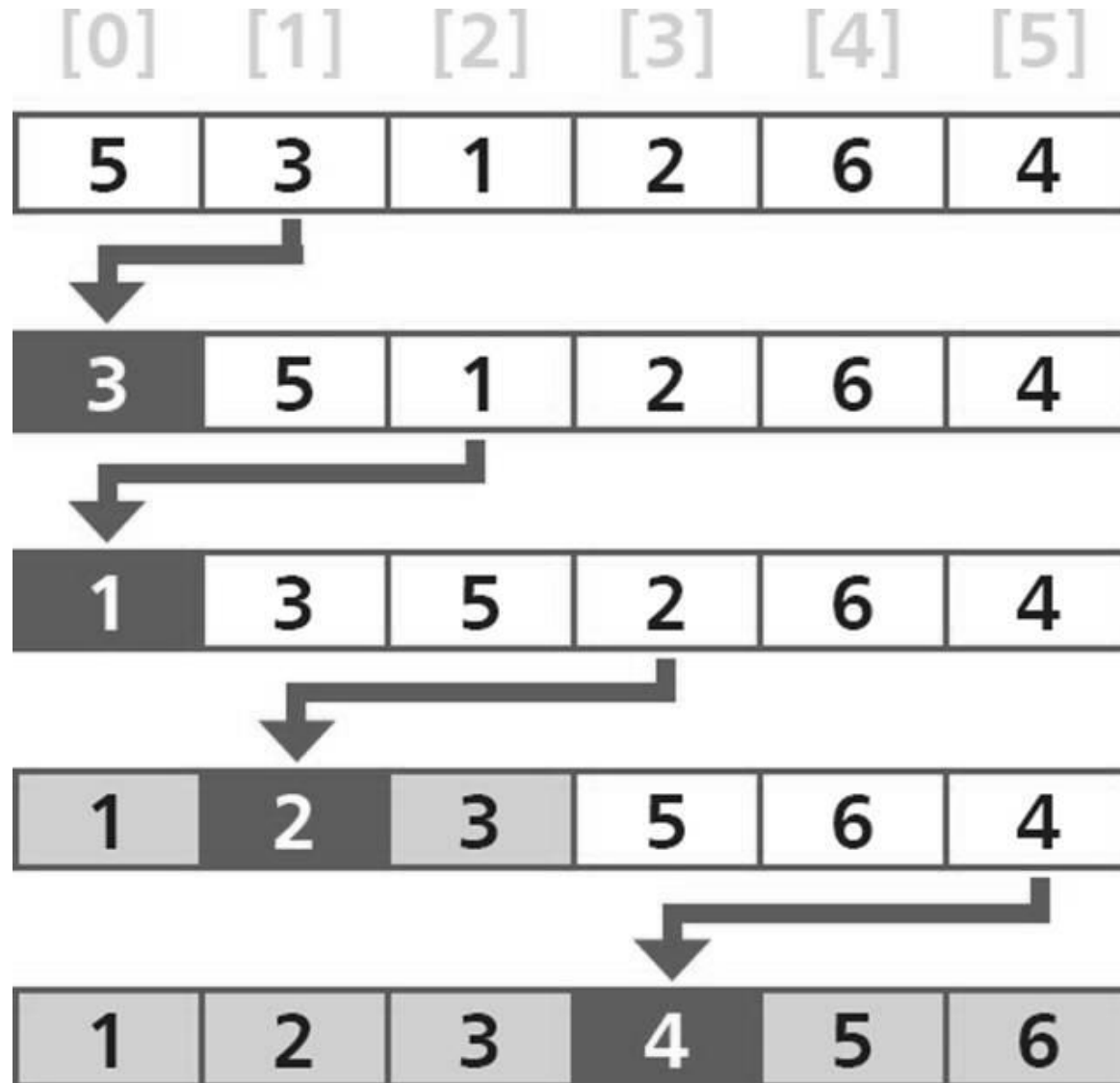


Step 3



Step 4

Insertion Sort



Selection Sort - Idea

- Idea:
 - Find the smallest element in the array
 - Exchange it with the element in the first position
 - Find the second smallest element and exchange it with the element in the second position
 - Continue until the array is sorted
- Disadvantage:
 - Running time depends only slightly on the amount of order in the file

Selection Sort

Alg.: SELECTION-SORT(A)

$n \leftarrow \text{length}[A]$

for $j \leftarrow 1$ **to** $n - 1$

do $\text{smallest} \leftarrow j$

for $i \leftarrow j + 1$ **to** n

do if $A[i] < A[\text{smallest}]$

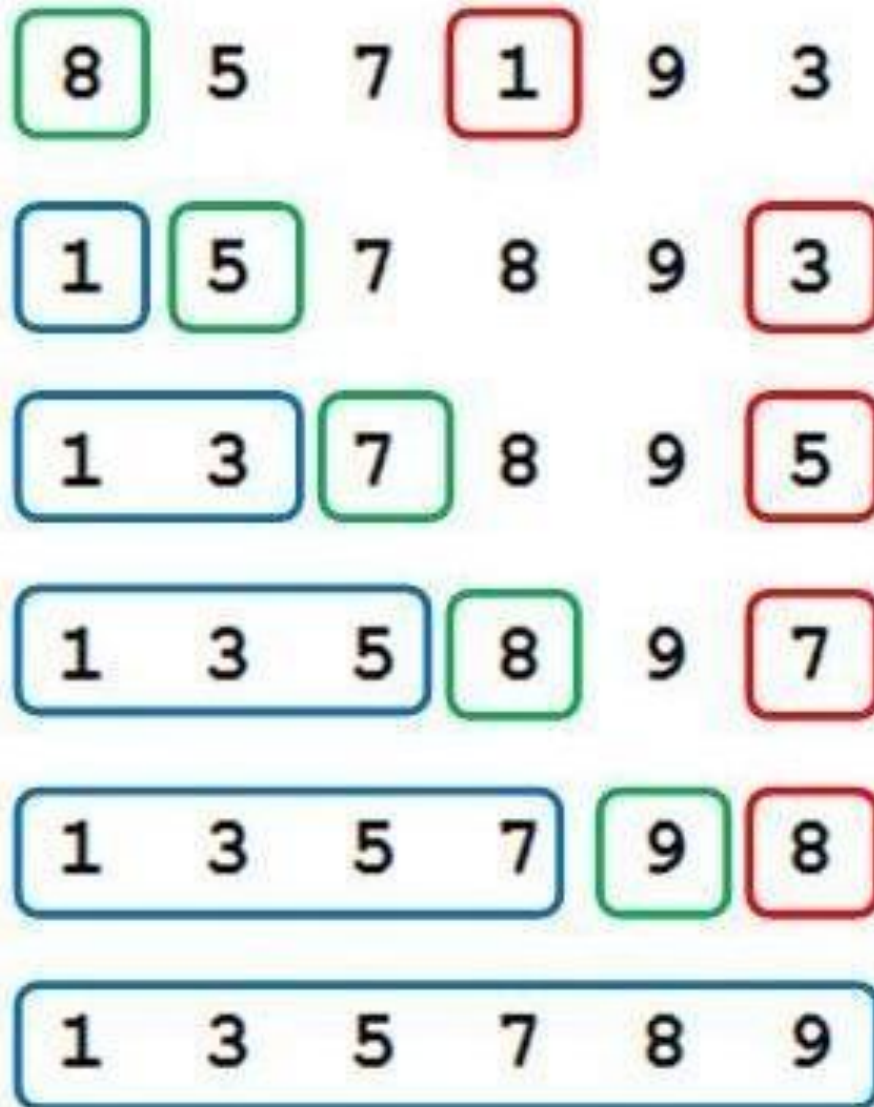
then $\text{smallest} \leftarrow i$

end for

 exchange $A[j] \leftrightarrow A[\text{smallest}]$

end for

Selection Sort



Sorted List.

Merge Sort - Basics

- To sort an array $A[p \dots r]$:
- **Divide**
 - Divide the n -element sequence to be sorted into two subsequences of $n/2$ elements each
- **Conquer**
 - Sort the subsequences recursively using merge sort
 - When the size of the sequences is 1 there is nothing more to do
- **Combine**
 - Merge the two sorted subsequences

Merge Sort – Main Algorithm

Alg.: MERGE-SORT(A, p, r)

if $p < r$

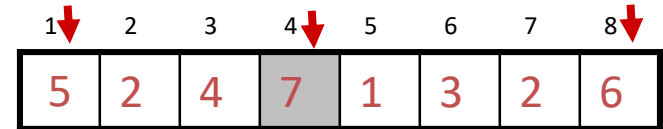
then $q \leftarrow \lfloor (p + r) / 2 \rfloor$

MERGE-SORT(A, p, q)

MERGE-SORT(A, q + 1, r)

MERGE(A, p, q, r)

- **Initial call:** MERGE-SORT(A, 1, n)



▷ Check for base case

▷ Divide

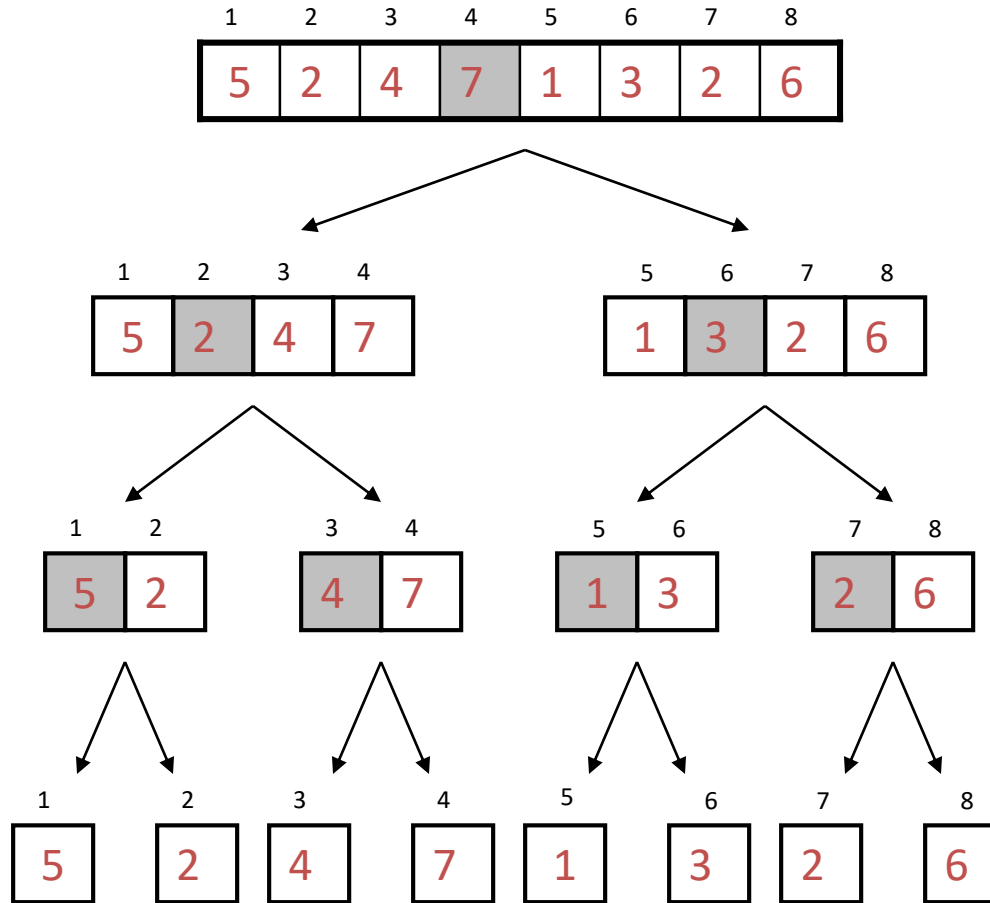
▷ Conquer

▷ Conquer

▷ Combine

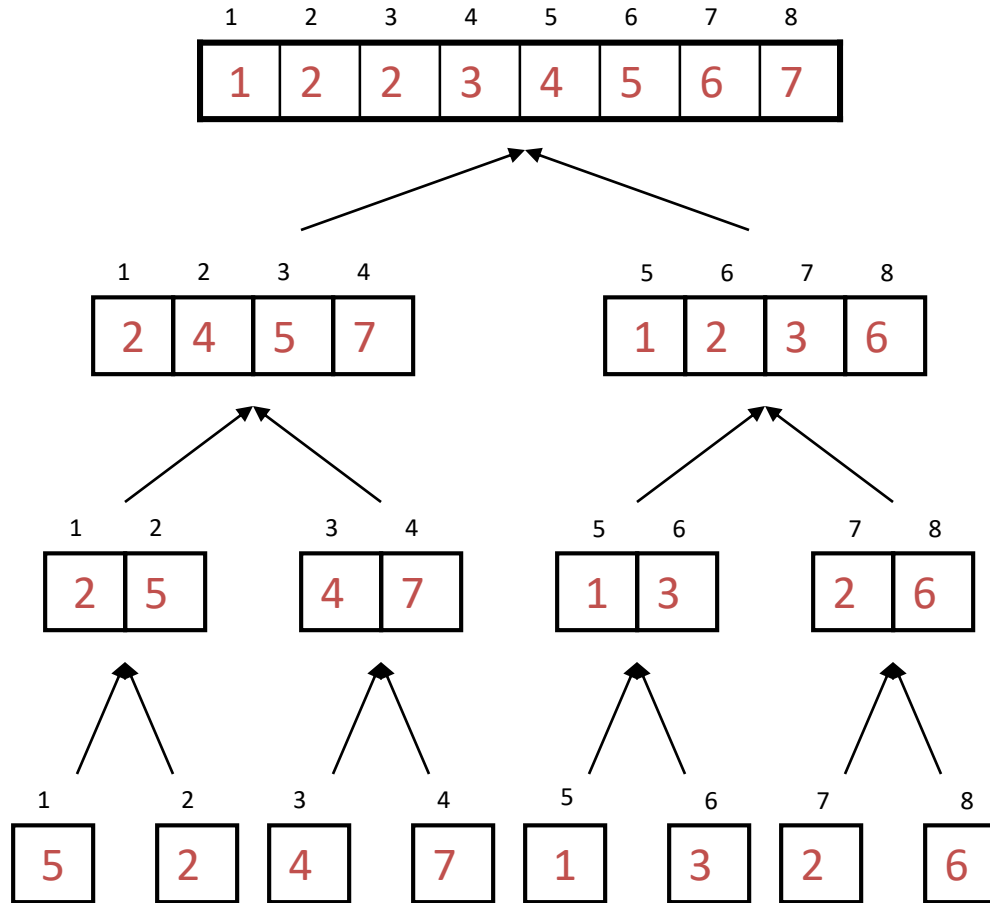
Merge Sort – Divide Process

Divide

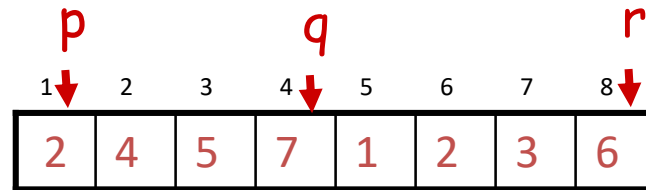

$$q = 4$$

Merge Sort – Conquer & Merge Process

Conquer
and
Merge



Merge Sort – Merge Process



- **Input:** Array A and indices p, q, r such that $p \leq q < r$
 - Subarrays $A[p..q]$ and $A[q+1..r]$ are sorted
- **Output:** One single sorted subarray $A[p..r]$

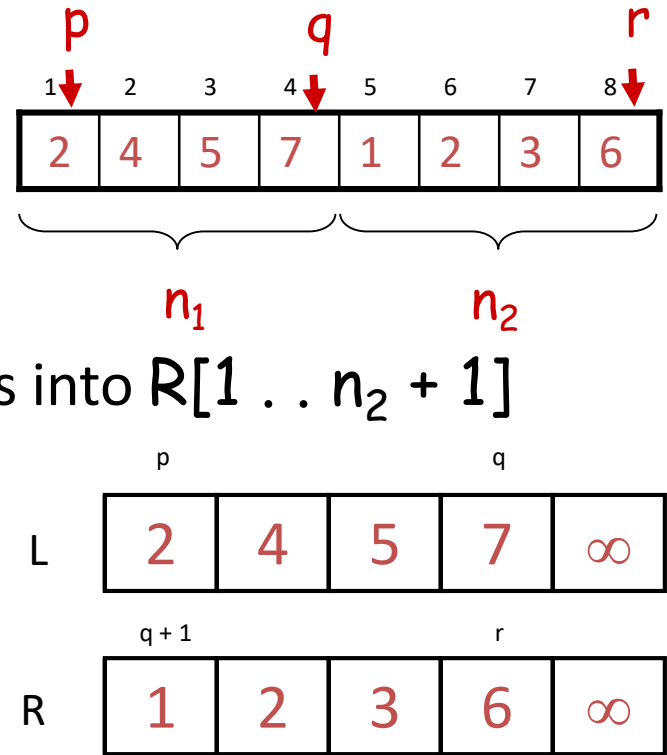
Merge Sort – Merge Process

- Idea for merging:
 - Two piles of sorted cards
 - Choose the smaller of the two top cards
 - Remove it and place it in the output pile
 - Repeat the process until one pile is empty
 - Take the remaining input pile and place it face-down onto the output pile

Merge Sort – Merging Algorithm

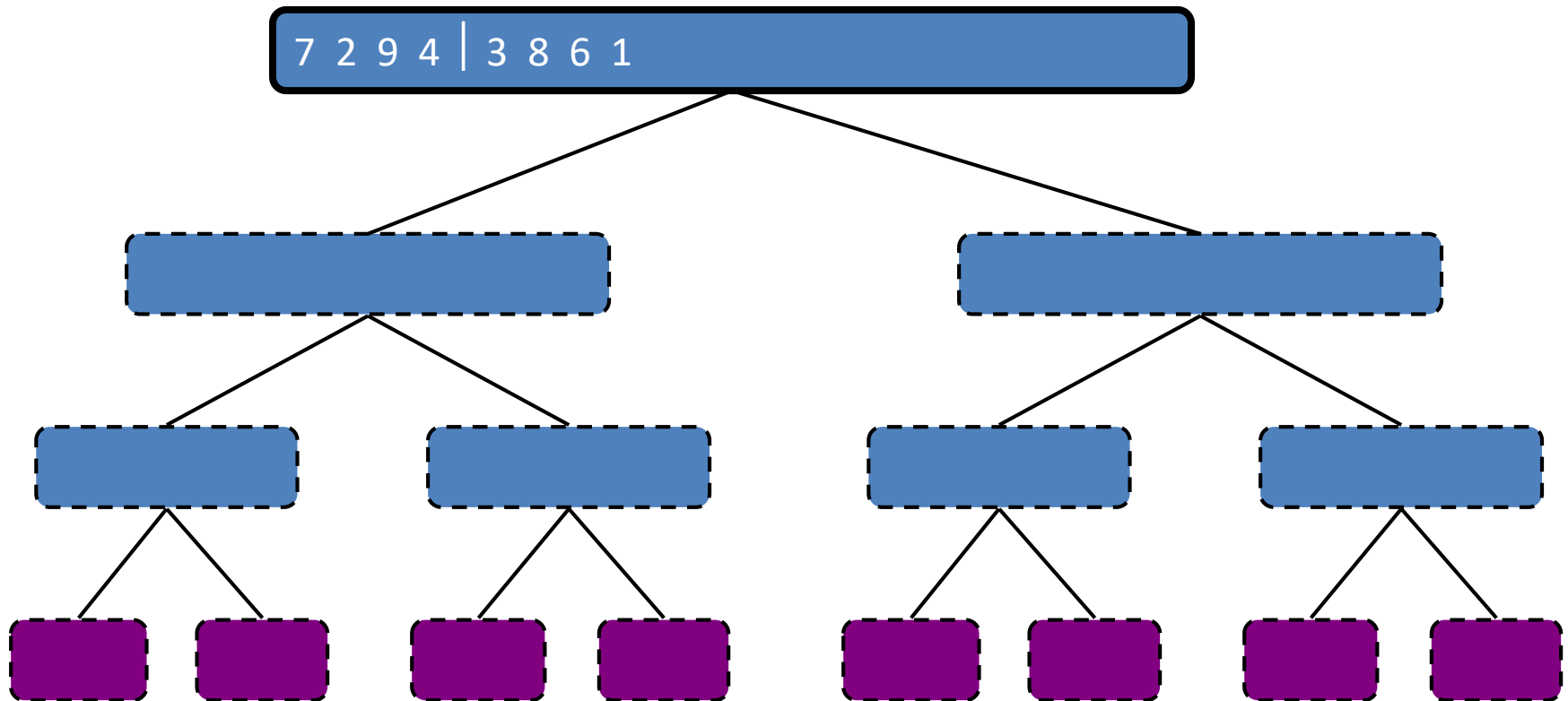
Alg.: MERGE(A, p, q, r)

1. Compute n_1 and n_2
2. Copy the first n_1 elements into $L[1 \dots n_1 + 1]$ and the next n_2 elements into $R[1 \dots n_2 + 1]$
 1. $L[n_1 + 1] \leftarrow \infty$; $R[n_2 + 1] \leftarrow \infty$
 2. $i \leftarrow 1$; $j \leftarrow 1$
 3. **for** $k \leftarrow p$ **to** r
 4. **do if** $L[i] \leq R[j]$
 5. **then** $A[k] \leftarrow L[i]$
 6. $i \leftarrow i + 1$
 7. **else** $A[k] \leftarrow R[j]$
 8. $j \leftarrow j + 1$



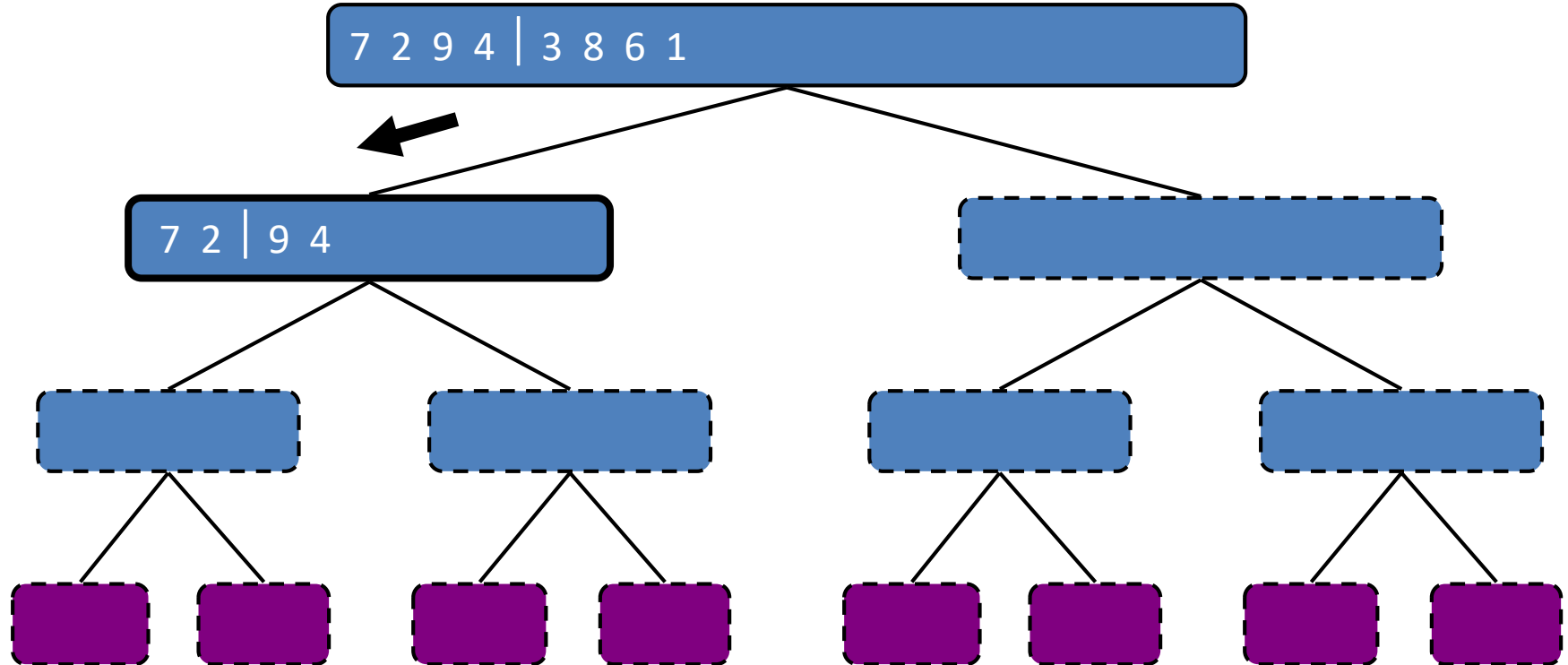
Merge Sort Example Step By Step

- Division



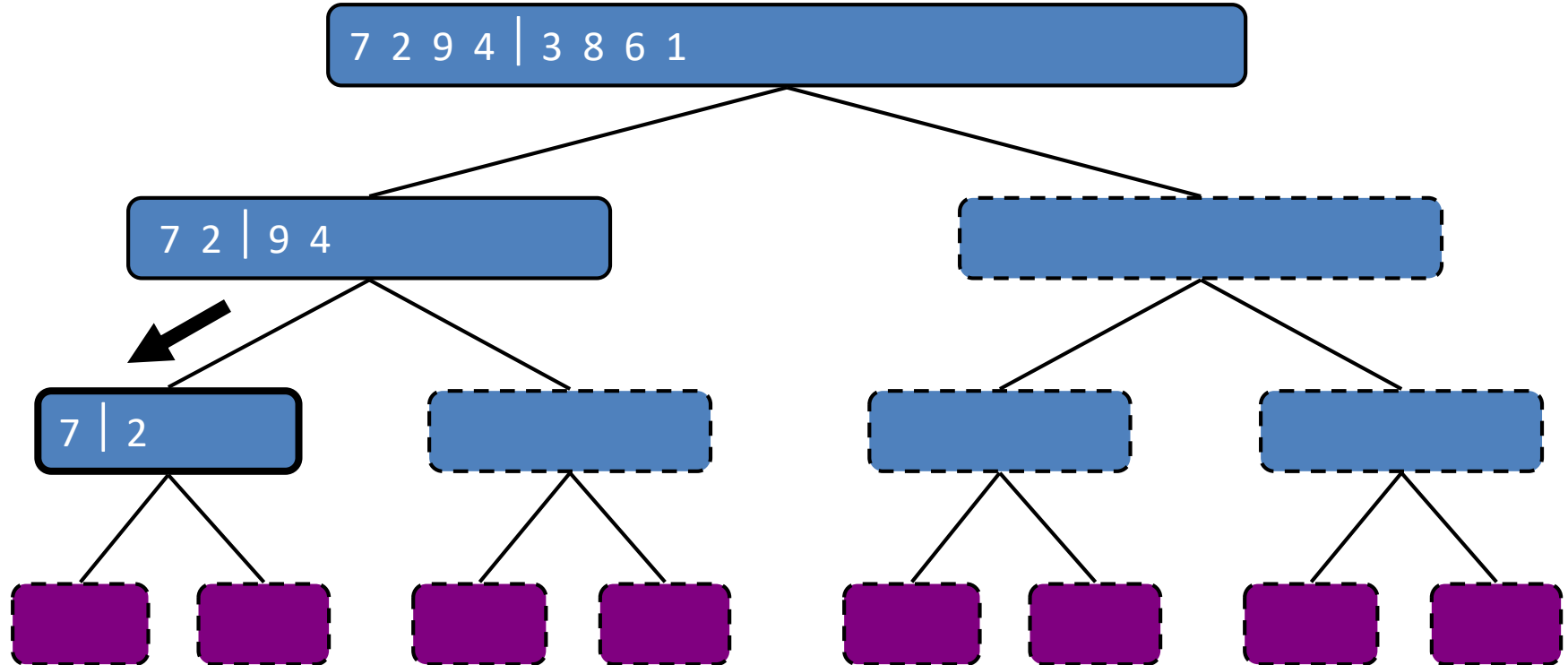
Merge Sort Example Step By Step

- Recursive call, Division



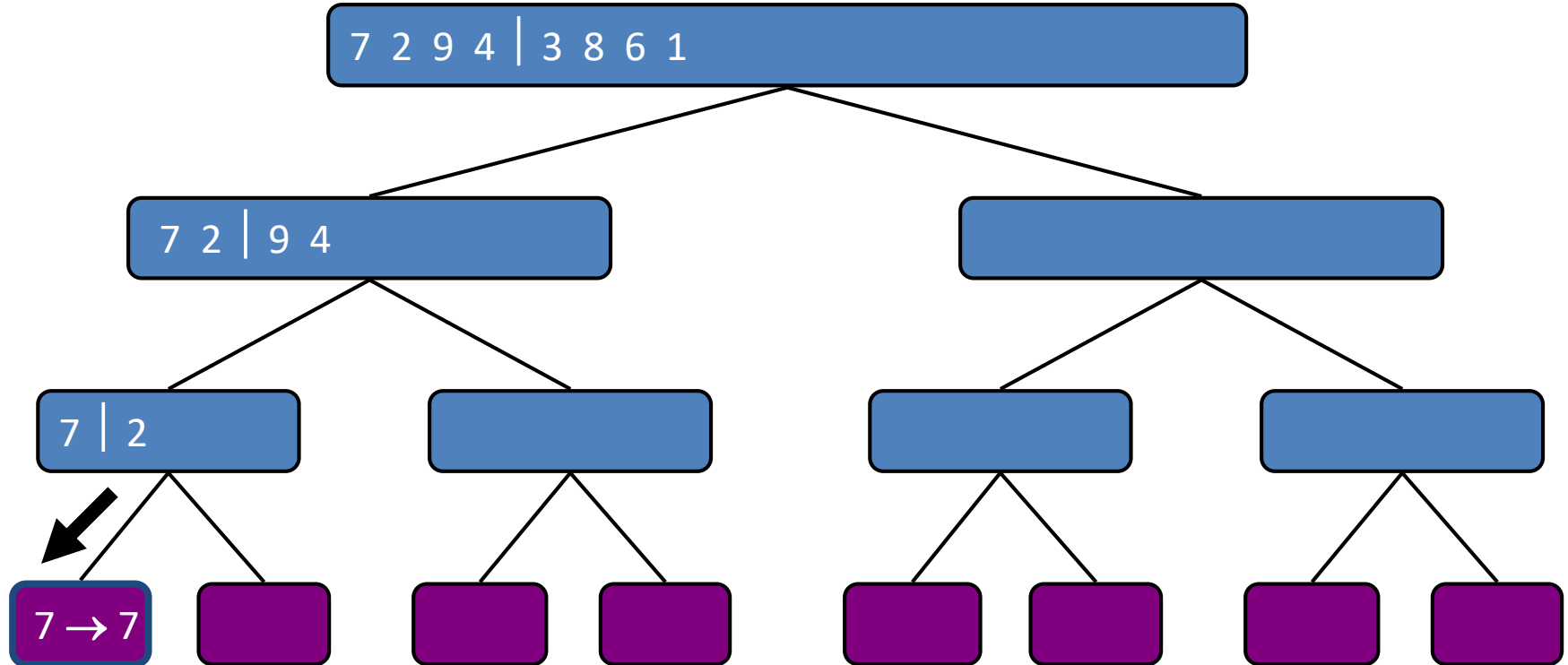
Merge Sort Example Step By Step

- Recursive call, Division



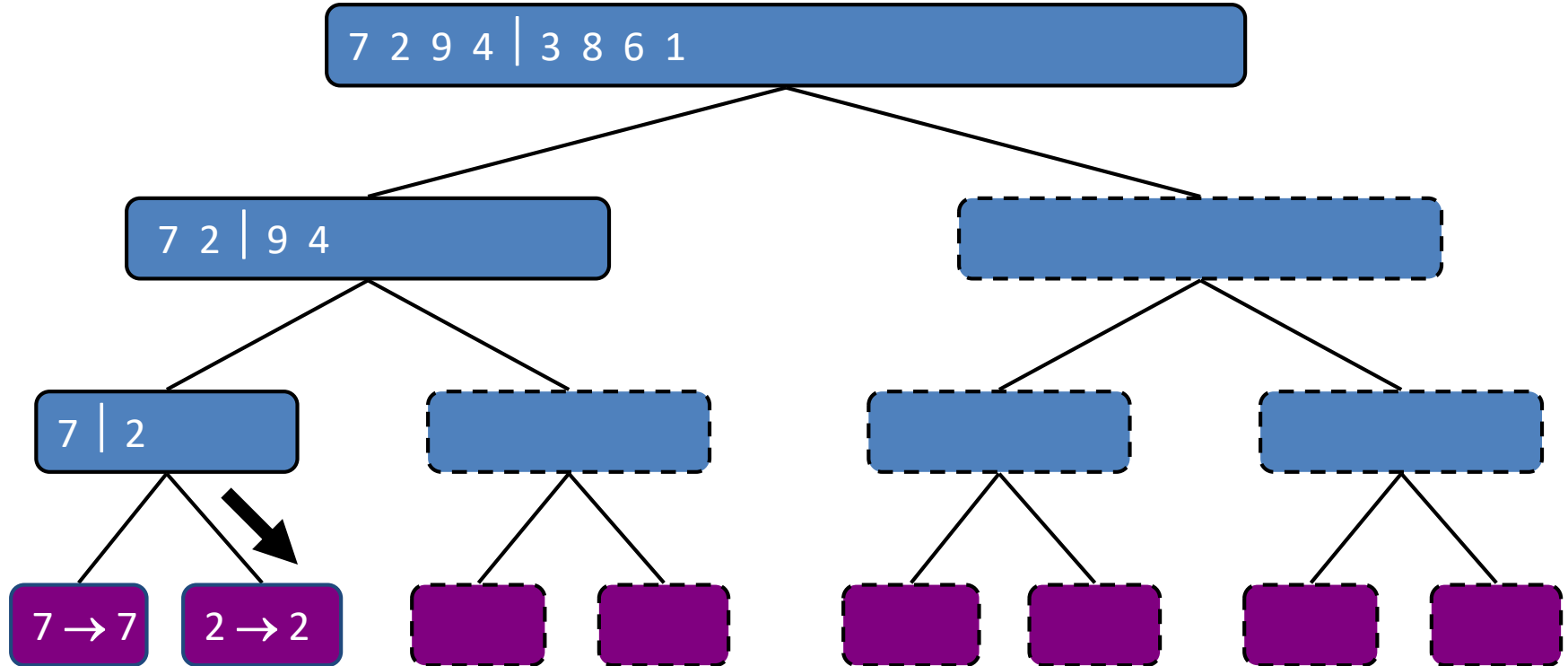
Merge Sort Example Step By Step

- Recursive call, base case



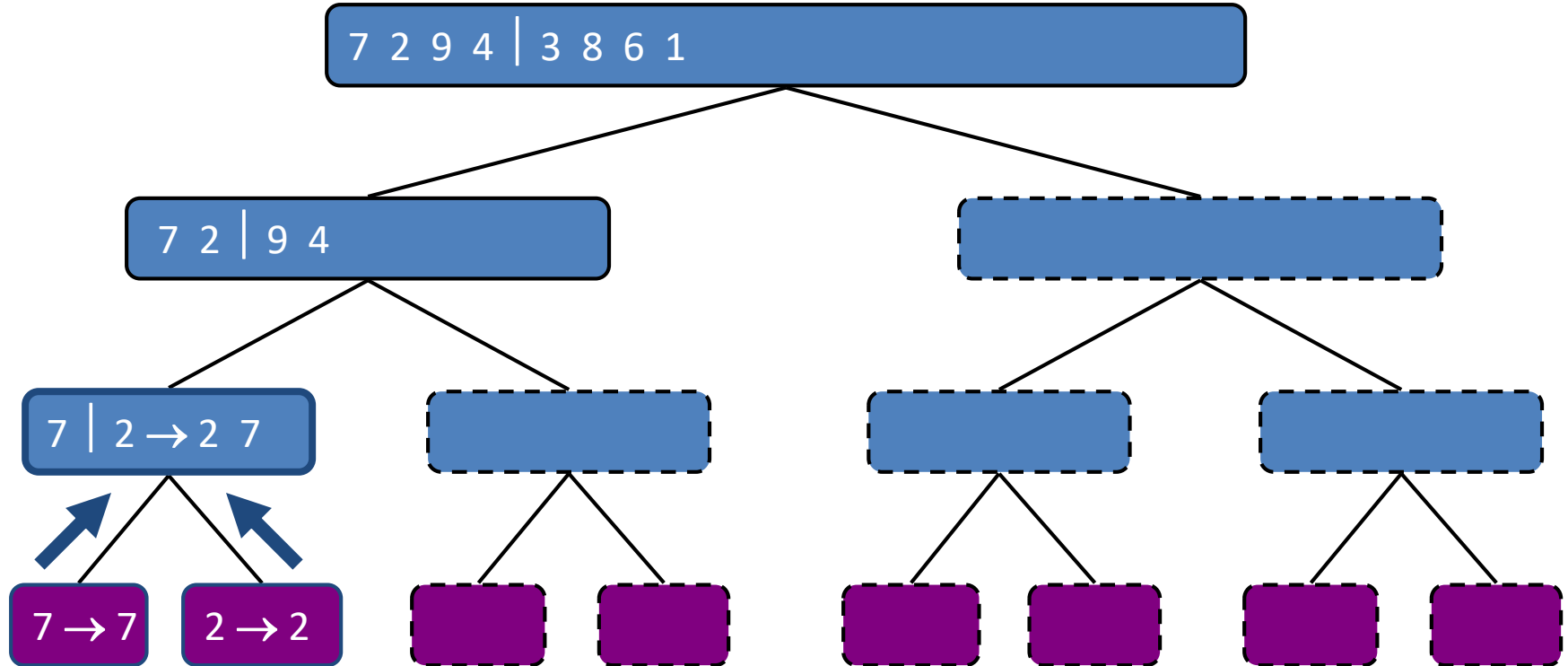
Merge Sort Example Step By Step

- Recursive call, base case



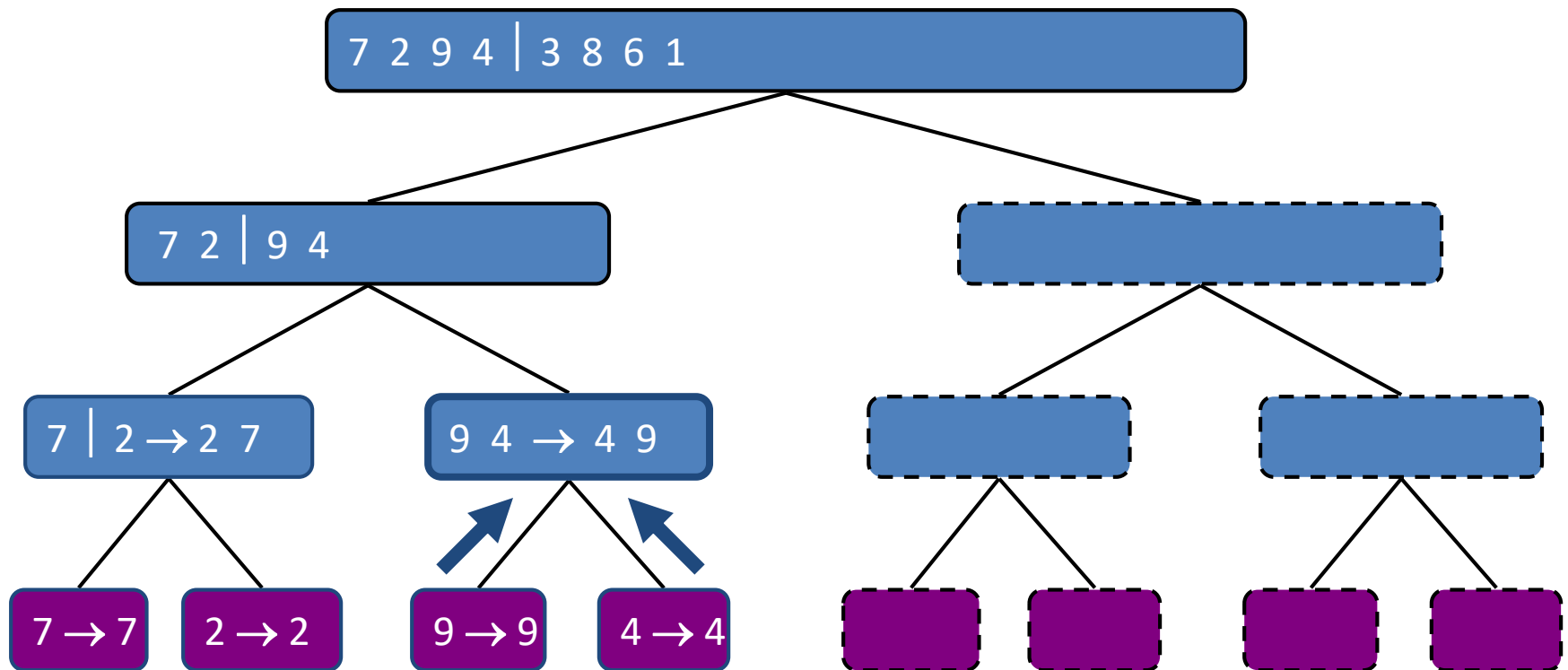
Merge Sort Example Step By Step

- Merge



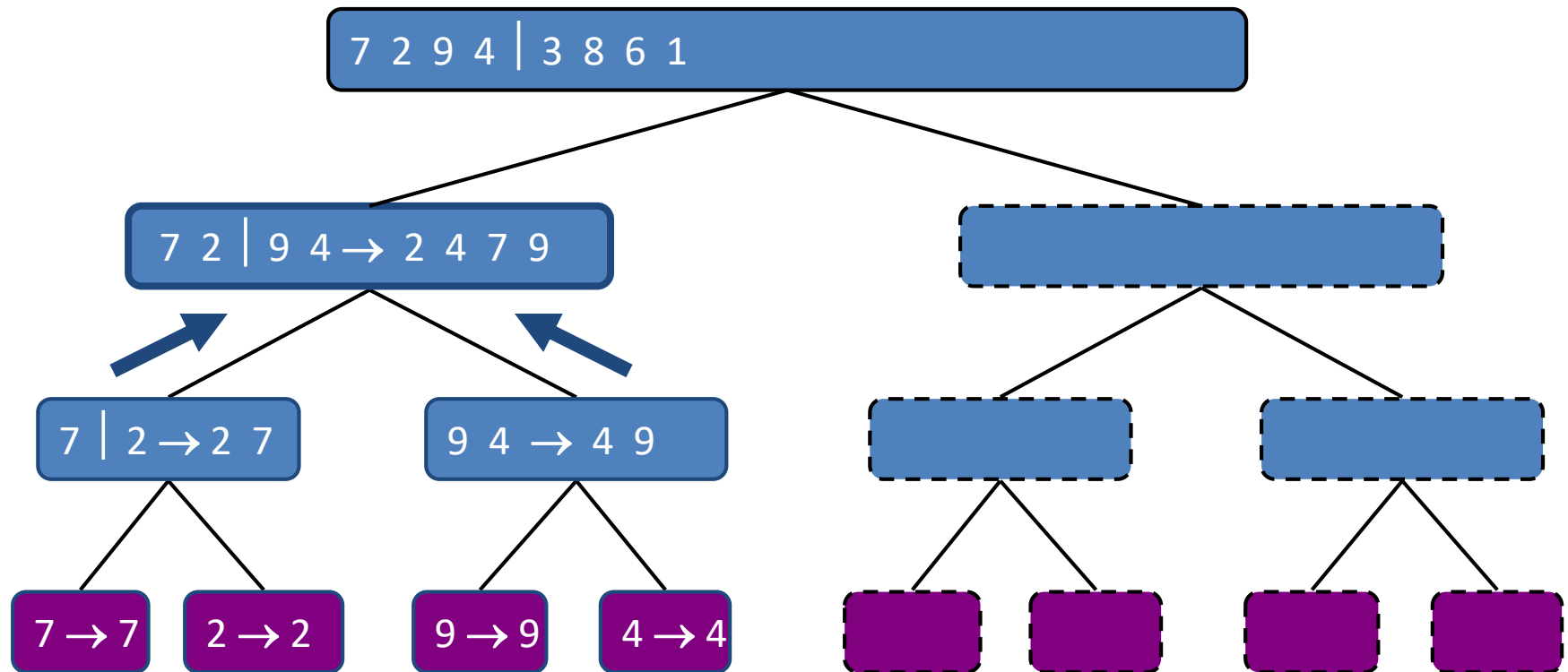
Merge Sort Example Step By Step

- Recursive call, ..., base case, merge



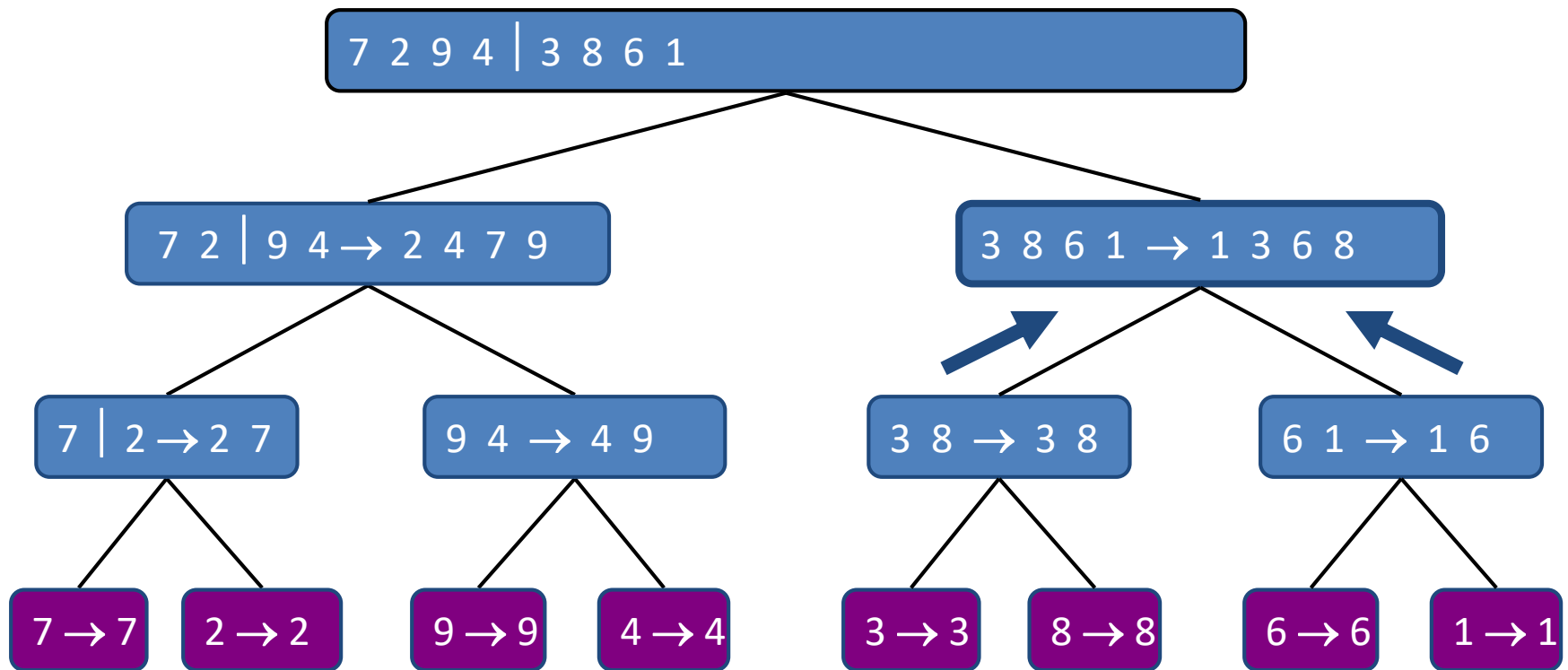
Merge Sort Example Step By Step

- Merge



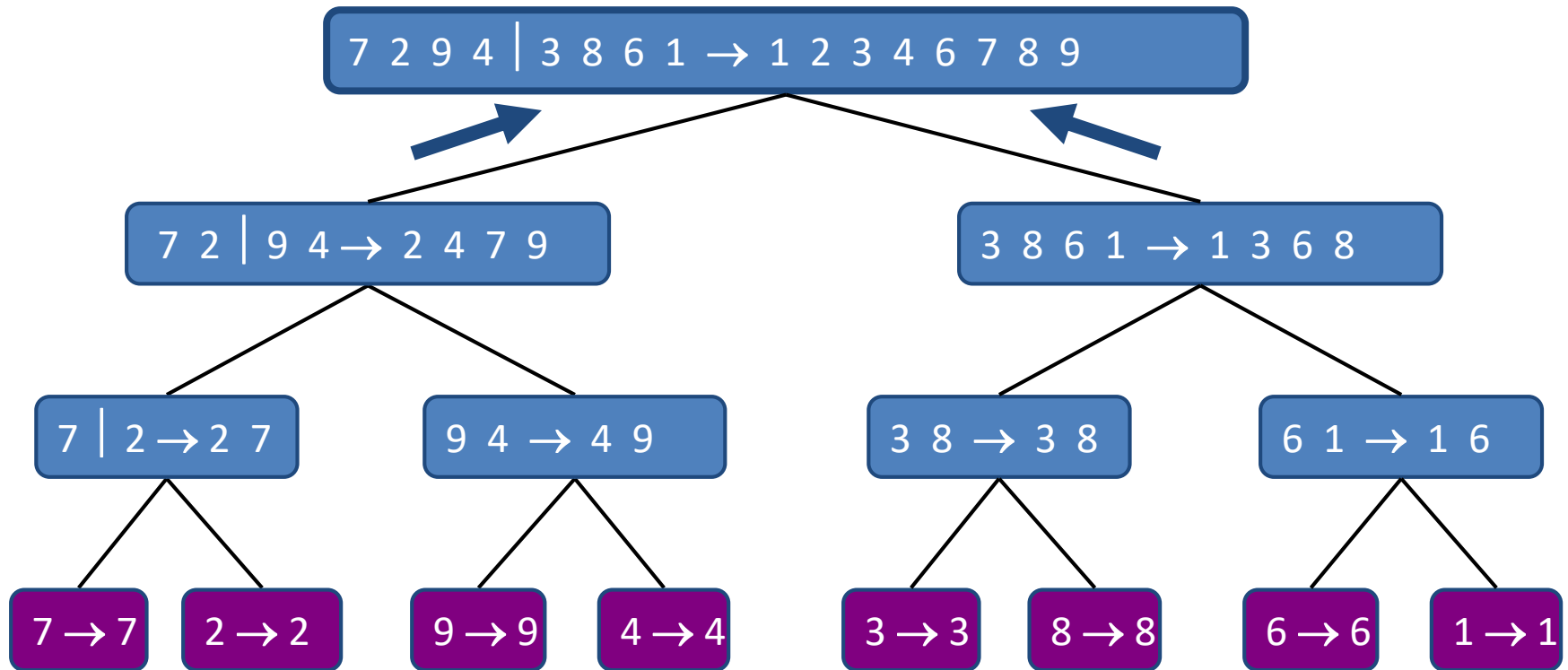
Merge Sort Example Step By Step

- Recursive call, ..., merge, merge



Merge Sort Example Step By Step

- Merge



Quick Sort - Basics

Divide:

partition $A[p..r]$ into two sub-arrays $A[p..q-1]$ and $A[q+1..r]$ such that each element of $A[p..q-1] \leq A[q]$, and each element of $A[q+1..r] \geq A[q]$. Compute q as part of this partitioning.

Conquer:

sort the sub-arrays $A[p..q-1]$ and $A[q+1..r]$ by recursive calls to QUICKSORT.

Combine:

the partitioning and recursive sorting leave us with a sorted $A[p..r]$ – **no work needed here**.

An obvious **difference** is that we do most of the work in the divide stage, with no work at the combine one.

Quick Sort - Algorithm

QuickSort (A, low, high)

If $\text{low} < \text{high}$

$q = \text{partition} (A, \text{low}, \text{high})$

 QuickSort (A, low, $q-1$)

 QuickSort (A, $q + 1$, high)

Partition (A, p, r)

1. $\text{pivot} = A [p]$

2. $i = p$

3. $j = r$

4. while ($i < j$)

 while ($A[i] \leq \text{pivot} \ \&\& \ i < r$)

$i = i + 1$;

 while ($A[j] > \text{pivot} \ \&\& \ j > p$)

$j = j - 1$;

 if ($i < j$)

 swap ($A [i] , A [j]$);

 end while

5. swap ($A [j] , \text{pivot}$)

6. Return (j)

`quickSort(arr, 0, 5)`

0	1	2	3	4	5
6	5	9	12	3	4

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

0	1	2	3	4	5
6	5	9	12	3	4

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot= ?`

0	1	2	3	4	5
6	5	9	12	3	4

Partition Initialization...

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

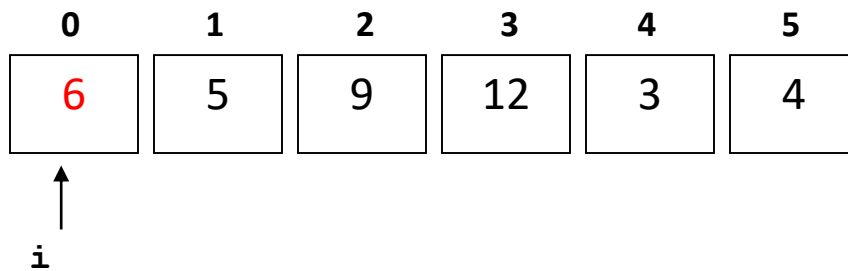
0	1	2	3	4	5
6	5	9	12	3	4

Partition Initialization...

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

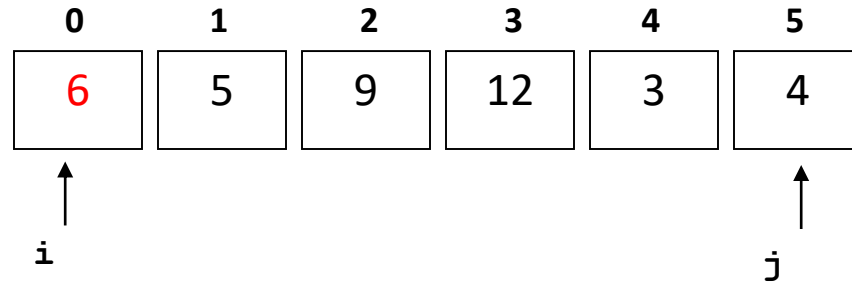


Partition Initialization...

```
quickSort(arr, 0, 5)
```

```
partition(arr, 0, 5)
```

pivot=6

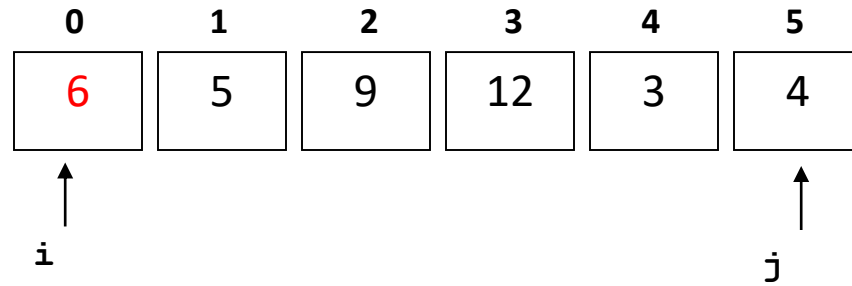


Partition Initialization...

```
quickSort(arr, 0, 5)
```

```
partition(arr, 0, 5)
```

pivot=6

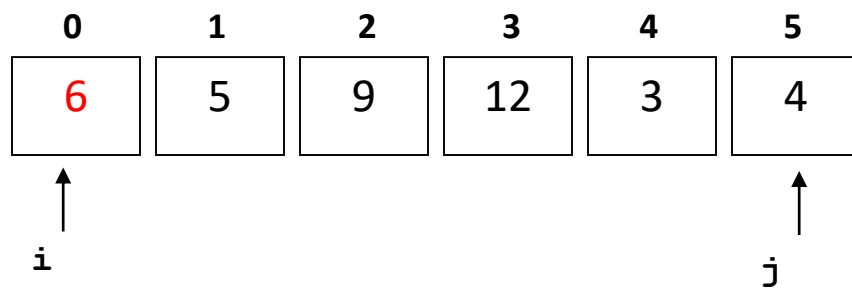


j moves to the i until
value that should be to left
of pivot...

```
quickSort(arr, 0, 5)
```

```
partition(arr, 0, 5)
```

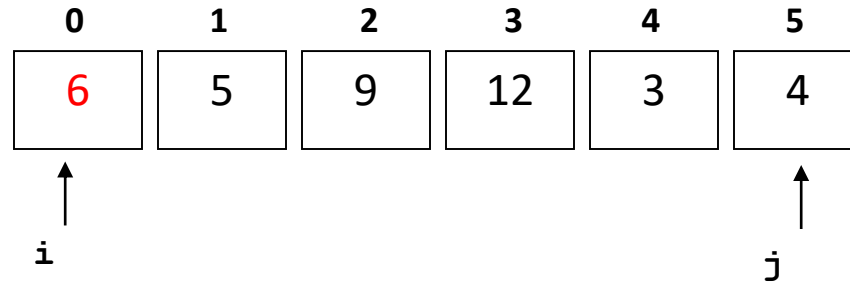
pivot=6



`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

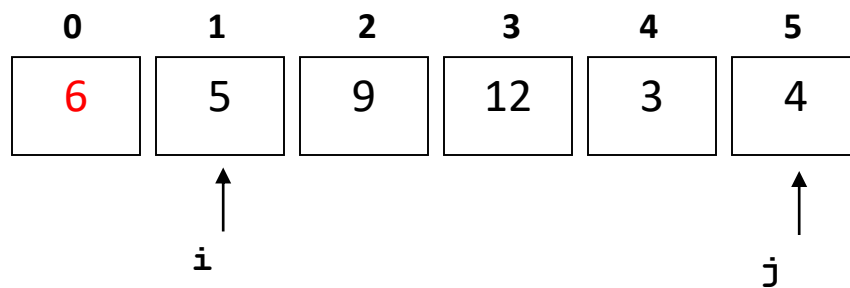


i moves to the j until
value that should be to right
of pivot...

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

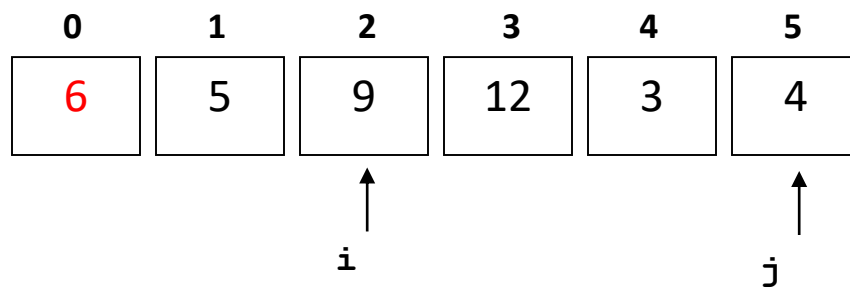
`pivot=6`



`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

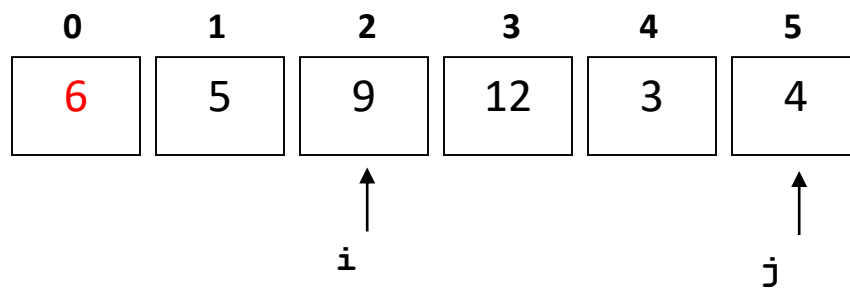
`pivot=6`



`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

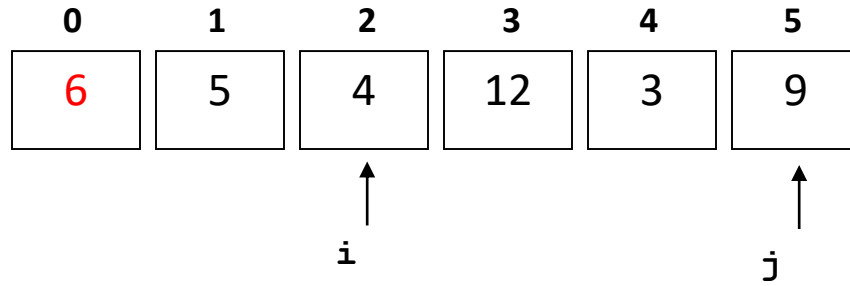


swap arr[i] and arr[j]

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

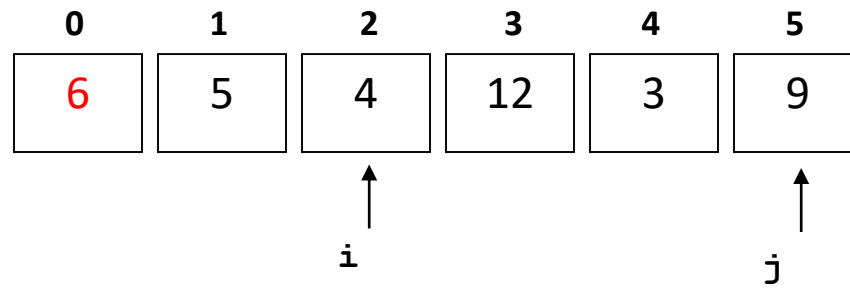


repeat j/i scan
UNTIL i & j cross

```
quickSort(arr, 0, 5)
```

```
partition(arr, 0, 5)
```

```
pivot=6
```

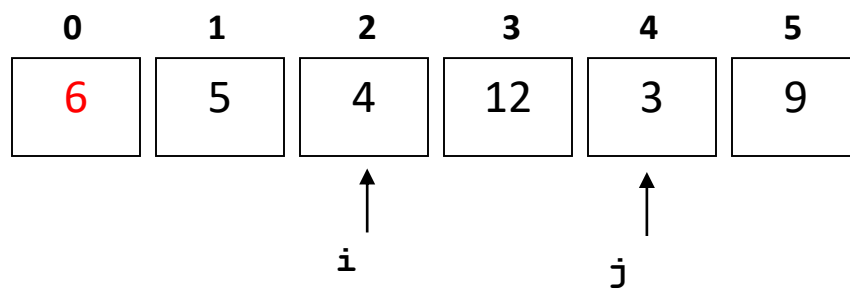


j moves to the i until
value that should be to left
of pivot...

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

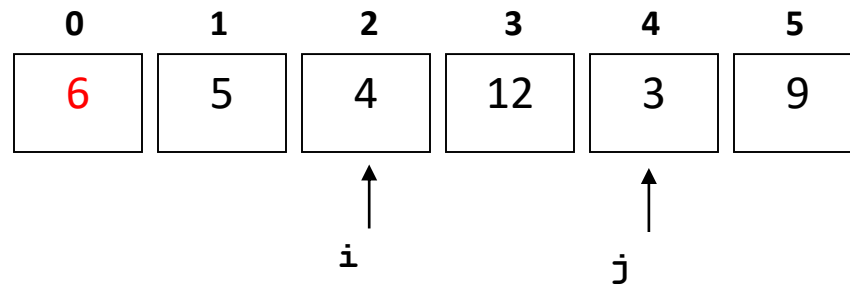
`pivot=6`



`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

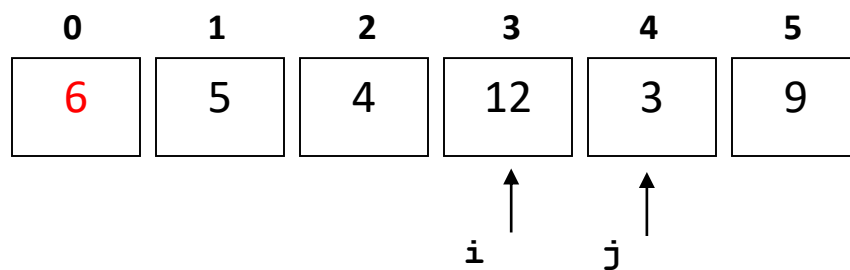


i moves to the *j* until
value that should be to right
of pivot...

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

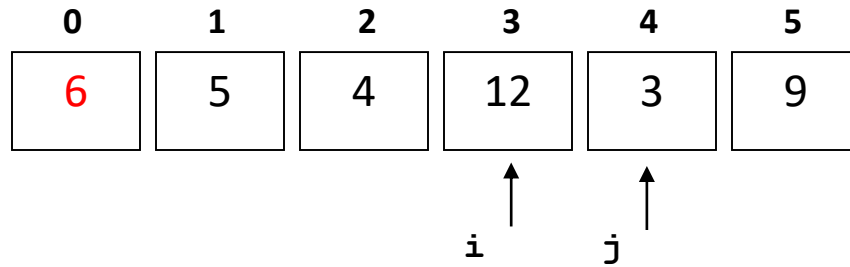
`pivot=6`



`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

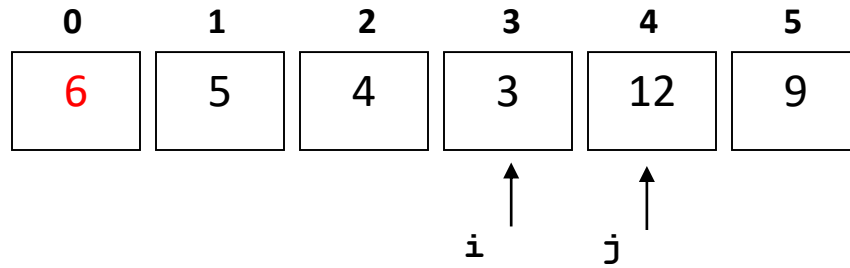


swap arr[i] and arr[j]

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

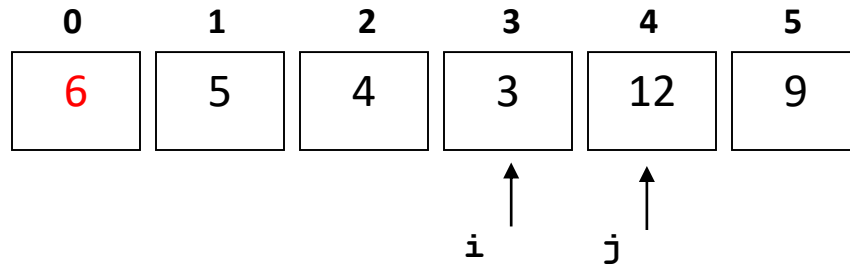


swap arr[*i*] and arr[*j*]

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

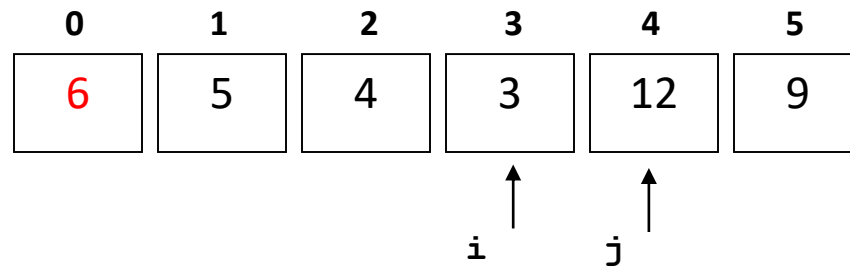


repeat j/i scan
UNTIL i & j cross

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

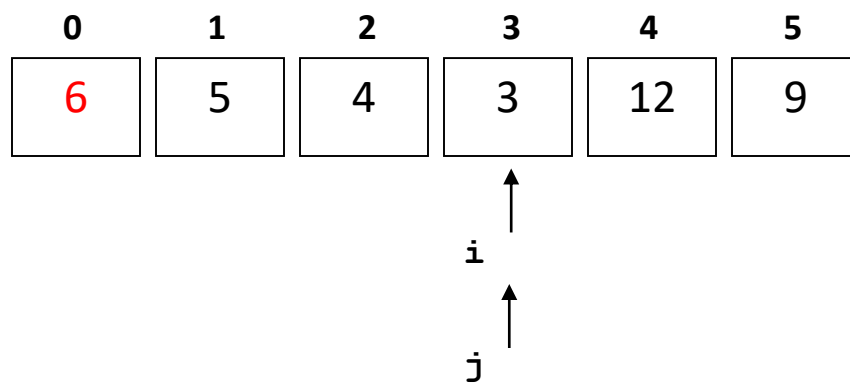


j moves to the i until
value that should be to left
of pivot...

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

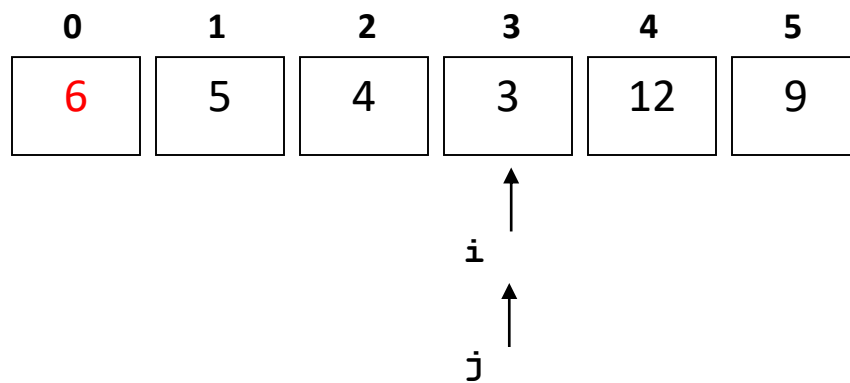
`pivot=6`



`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`

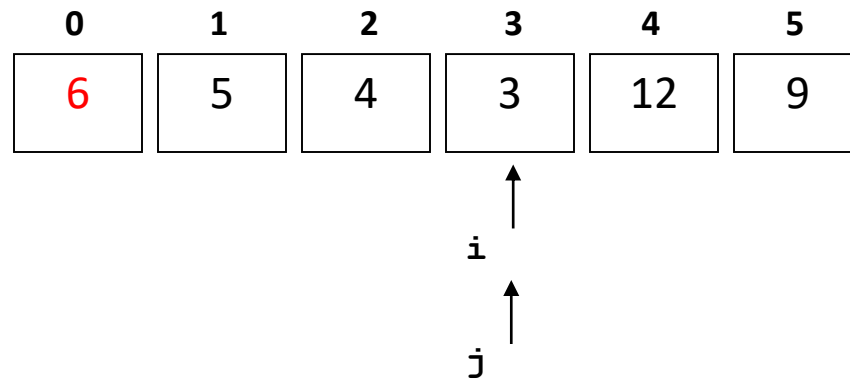


j & i CROSS!!!

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`



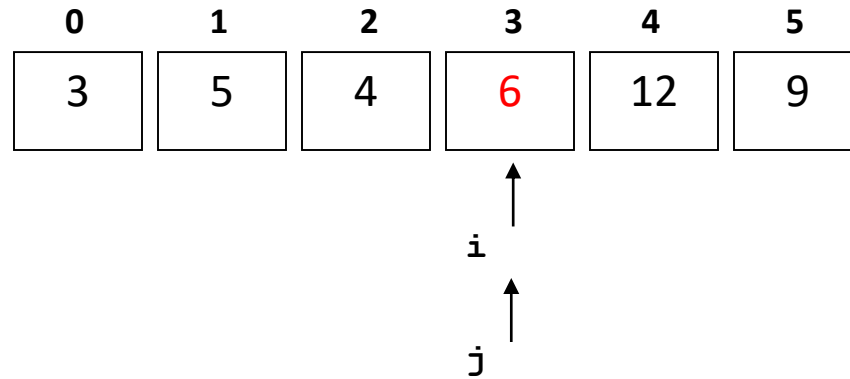
j & i CROSS!!!

1 - Swap pivot and arr[j]

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`



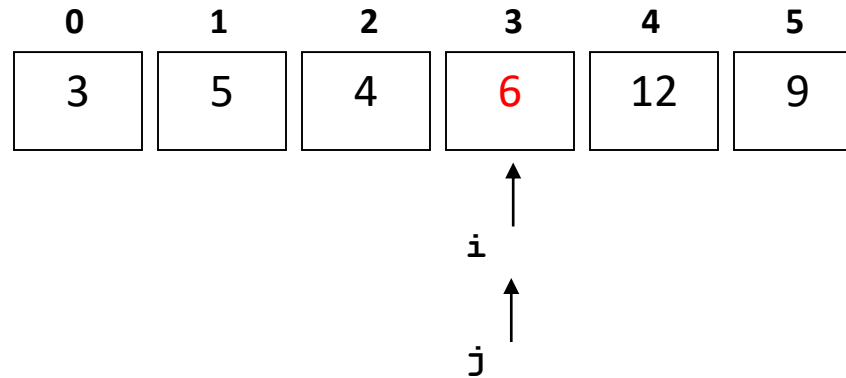
j & i CROSS!!!

1 - Swap pivot and arr[j]

`quickSort(arr, 0, 5)`

`partition(arr, 0, 5)`

`pivot=6`



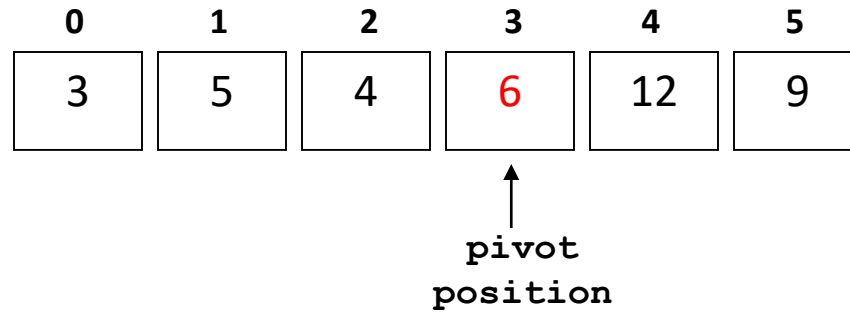
j & i CROSS!!!

1 - Swap pivot and arr[j]

2 - Return new location of pivot to caller

return 3

`quickSort(arr, 0, 5)`



Recursive calls to `quickSort()`
using partitioned array...

`quickSort(arr, 0, 5)`

0	1	2	3	4	5
3	5	4	6	12	9

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

0	1	2	3
3	5	4	6

4	5
12	9

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`

0	1	2	3
3	5	4	6

4	5
12	9

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`

0	1	2	3
3	5	4	6

4	5
12	9

Partition Initialization...

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`

0	1	2	3
3	5	4	6

4	5
12	9

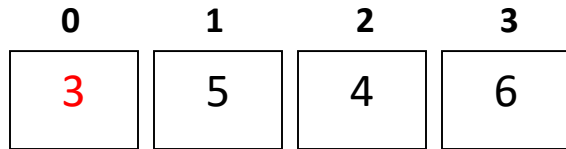
Partition Initialization...

`quickSort(arr, 0, 5)`

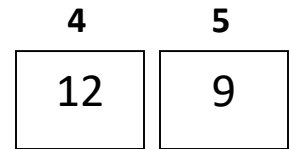
`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`



↑
`i`



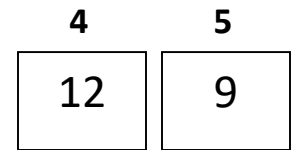
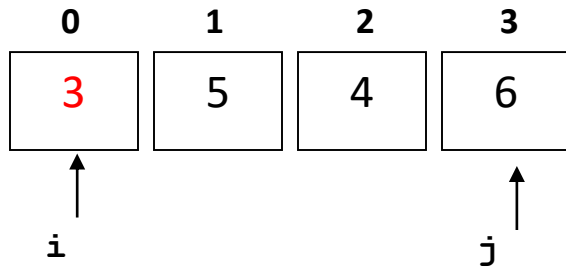
Partition Initialization...

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`



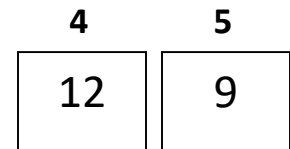
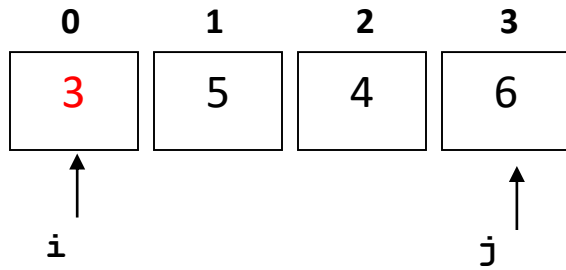
Partition Initialization...

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`



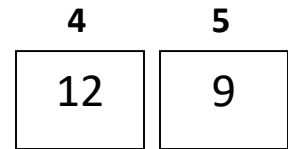
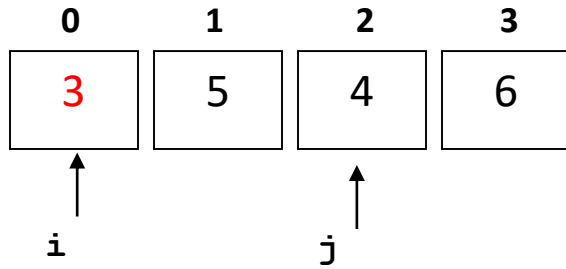
j moves to the i until
value that should be to left
of pivot...

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`

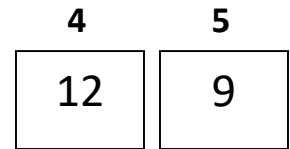
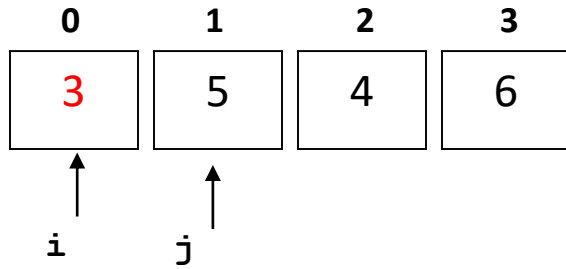


`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`

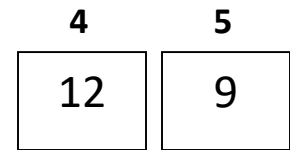
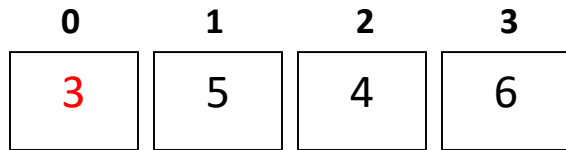


`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

`partition(arr, 0, 3)`



`i`



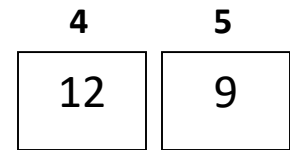
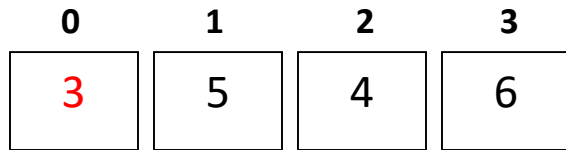
`j`

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

`partition(arr, 0, 3)`



`i`



`j`

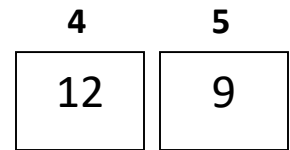
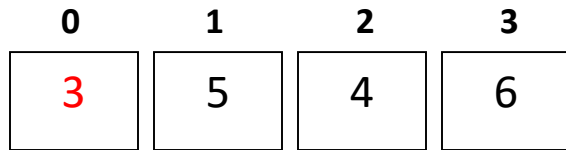
`j & i CROSS!!!`

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`



`i`



`j`

`j & i CROSS!!!`

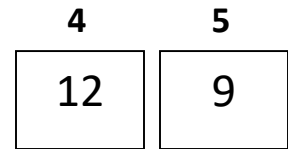
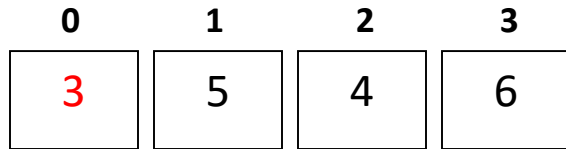
1 - Swap pivot and `arr[j]`

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`partition(arr, 0, 3)`

`quickSort(arr, 4, 5)`



`i`



`j`

`j & i CROSS!!!`

1 - Swap pivot and `arr[j]`

2 - Return new location of pivot to caller

`return 0`

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

0	1	2	3
3	5	4	6

4	5
12	9

Recursive calls to `quickSort()`
using partitioned array...

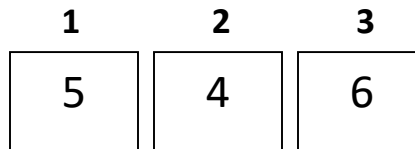
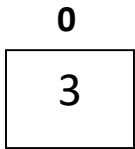
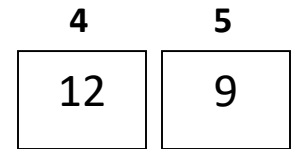
`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

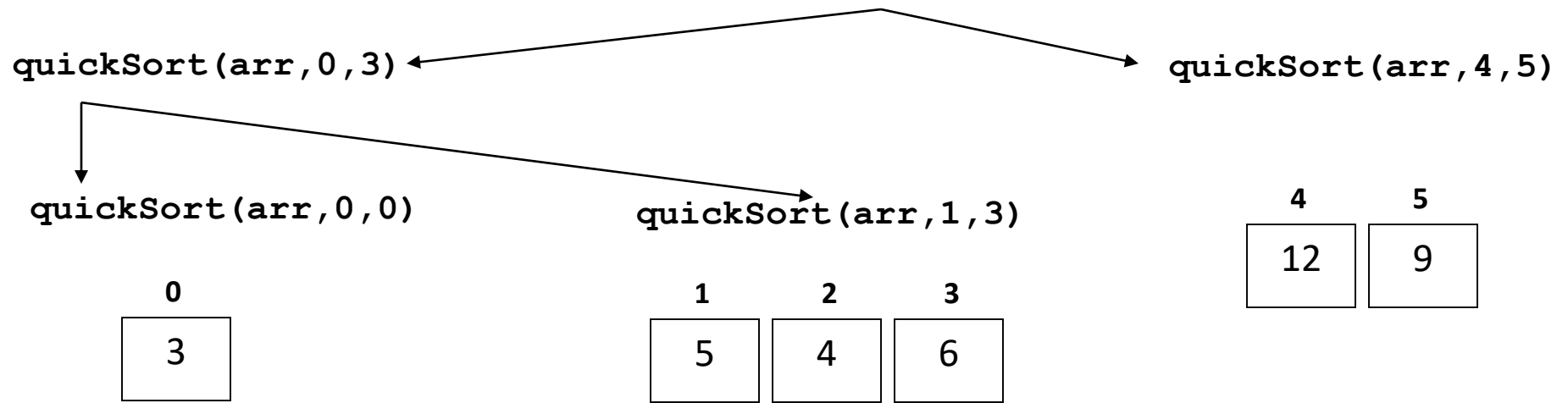
`quickSort(arr, 4, 5)`

`quickSort(arr, 0, 0)`

`quickSort(arr, 1, 3)`



`quickSort(arr, 0, 5)`



Base case triggered...
halting recursion.

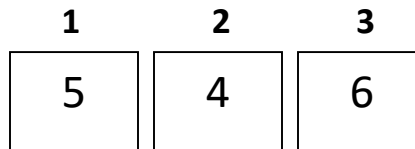
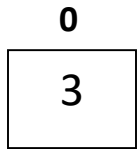
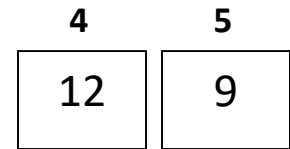
`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

`quickSort(arr, 0, 0)`

`quickSort(arr, 1, 3)`



Base Case: Return

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

`quickSort(arr, 0, 0)`

`quickSort(arr, 1, 3)`
`partition(arr, 1, 3)`

0
3

1 2 3
5 4 6

4 5
12 9

Partition Initialization...

`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

`quickSort(arr, 0, 0)`

`quickSort(arr, 1, 3)`
`partition(arr, 1, 3)`

0
3

1 2 3
5 4 6

4 5
12 9

Partition Initialization...

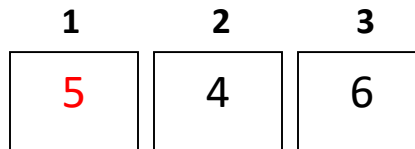
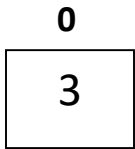
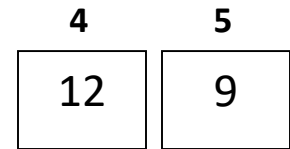
`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

`quickSort(arr, 0, 0)`

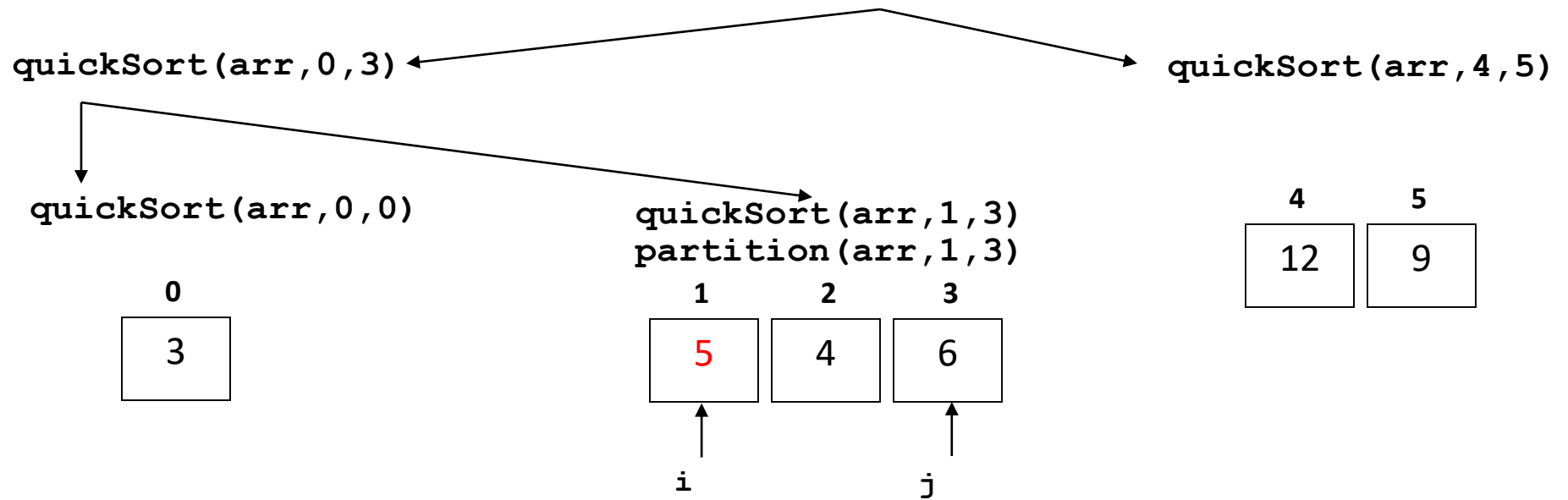
`quickSort(arr, 1, 3)`
`partition(arr, 1, 3)`



↑
i

Partition Initialization...

`quickSort(arr, 0, 5)`



j moves to the i until
value that should be to left
of pivot...

quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)

quickSort(arr, 0, 0)

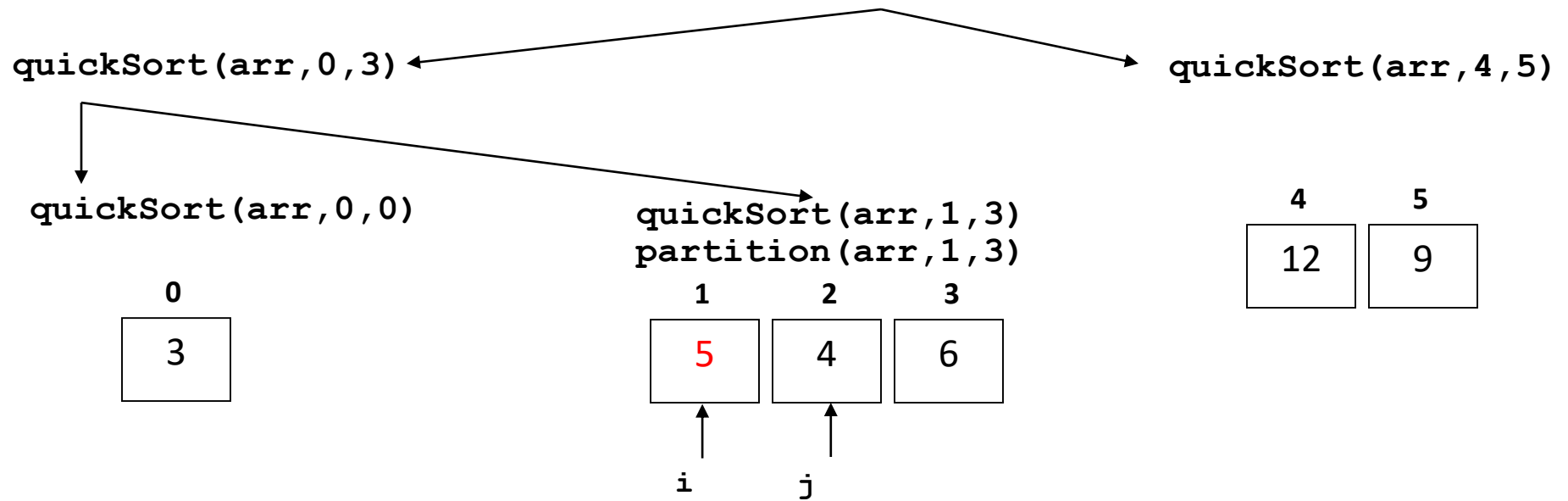
0
3

quickSort(arr, 1, 3)
partition(arr, 1, 3)

1 2 3
5 4 6
i j

4 5
12 9

`quickSort(arr, 0, 5)`



i moves to the j until
value that should be to right
of pivot...

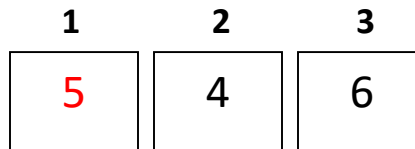
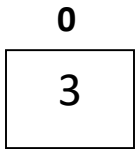
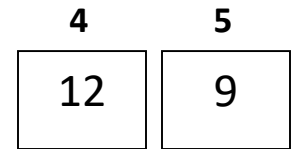
`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

`quickSort(arr, 0, 0)`

`quickSort(arr, 1, 3)`
`partition(arr, 1, 3)`



j
↑
i

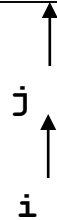
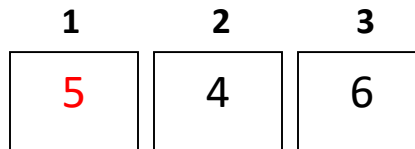
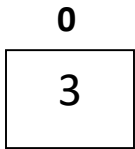
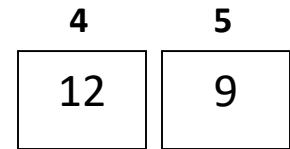
quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)

quickSort(arr, 0, 0)

quickSort(arr, 1, 3)
partition(arr, 1, 3)



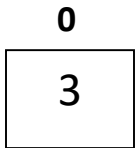
j & i CROSS!

`quickSort(arr, 0, 5)`

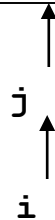
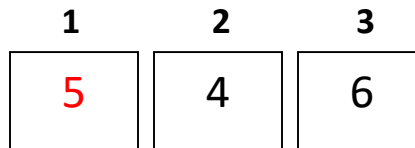
`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

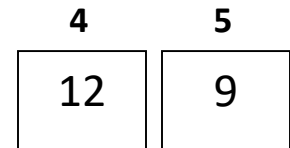
`quickSort(arr, 0, 0)`



`quickSort(arr, 1, 3)`
`partition(arr, 1, 3)`



j & i CROSS!
1- swap pivot and arr[j]

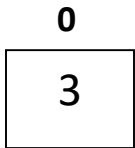


`quickSort(arr, 0, 5)`

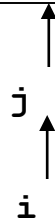
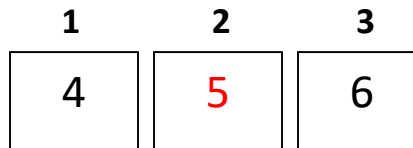
`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

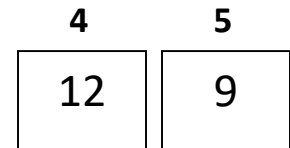
`quickSort(arr, 0, 0)`



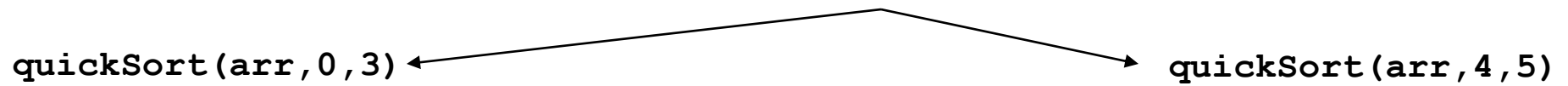
`quickSort(arr, 1, 3)`
`partition(arr, 1, 3)`



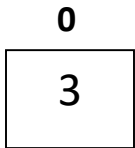
j & i CROSS!
1- swap pivot and arr[j]



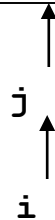
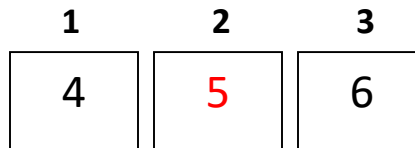
`quickSort(arr, 0, 5)`



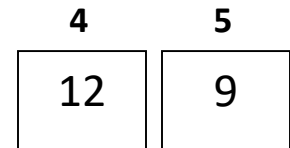
`quickSort(arr, 0, 0)`



`quickSort(arr, 1, 3)`
`partition(arr, 1, 3)`



j & i CROSS!
1- swap pivot and arr[j]
2 – return new position of pivot



return 2

quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)

quickSort(arr, 0, 0)

quickSort(arr, 1, 3)

4

5

12

9

0

3

quickSort(arr, 1, 2)

quickSort(arr, 3, 3)

1

2

4

5

3

6

quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)

quickSort(arr, 0, 0)

quickSort(arr, 1, 3)

4

5

12

9

0

3

quickSort(arr, 1, 2)
partition(arr, 1, 2)

quickSort(arr, 3, 3)

1

2

4

5

3

6

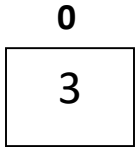
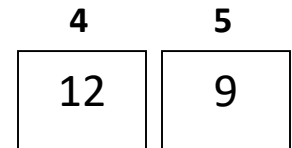
`quickSort(arr, 0, 5)`

`quickSort(arr, 0, 3)`

`quickSort(arr, 4, 5)`

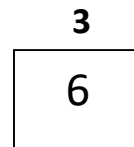
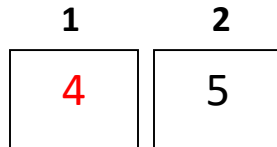
`quickSort(arr, 0, 0)`

`quickSort(arr, 1, 3)`



`quickSort(arr, 1, 2)`

`quickSort(arr, 3, 3)`



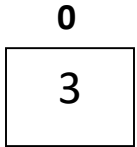
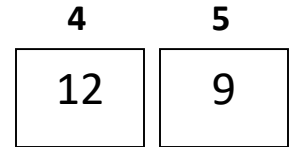
quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)

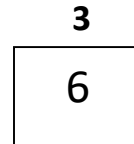
quickSort(arr, 0, 0)

quickSort(arr, 1, 3)



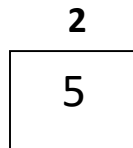
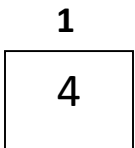
quickSort(arr, 1, 2)

quickSort(arr, 3, 3)



quickSort(arr, 1, 1)

quickSort(arr, 2, 2)



quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)
partition(arr, 4, 5)

quickSort(arr, 0, 0)

quickSort(arr, 1, 3)

4	5
12	9

0
3

quickSort(arr, 1, 2)

quickSort(arr, 3, 3)

3
6

quickSort(arr, 1, 1)

quickSort(arr, 2, 2)

1
4

2
5

quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)
partition(arr, 4, 5)

quickSort(arr, 0, 0)

quickSort(arr, 1, 3)

4	5
9	12

0
3

quickSort(arr, 1, 2)

quickSort(arr, 3, 3)

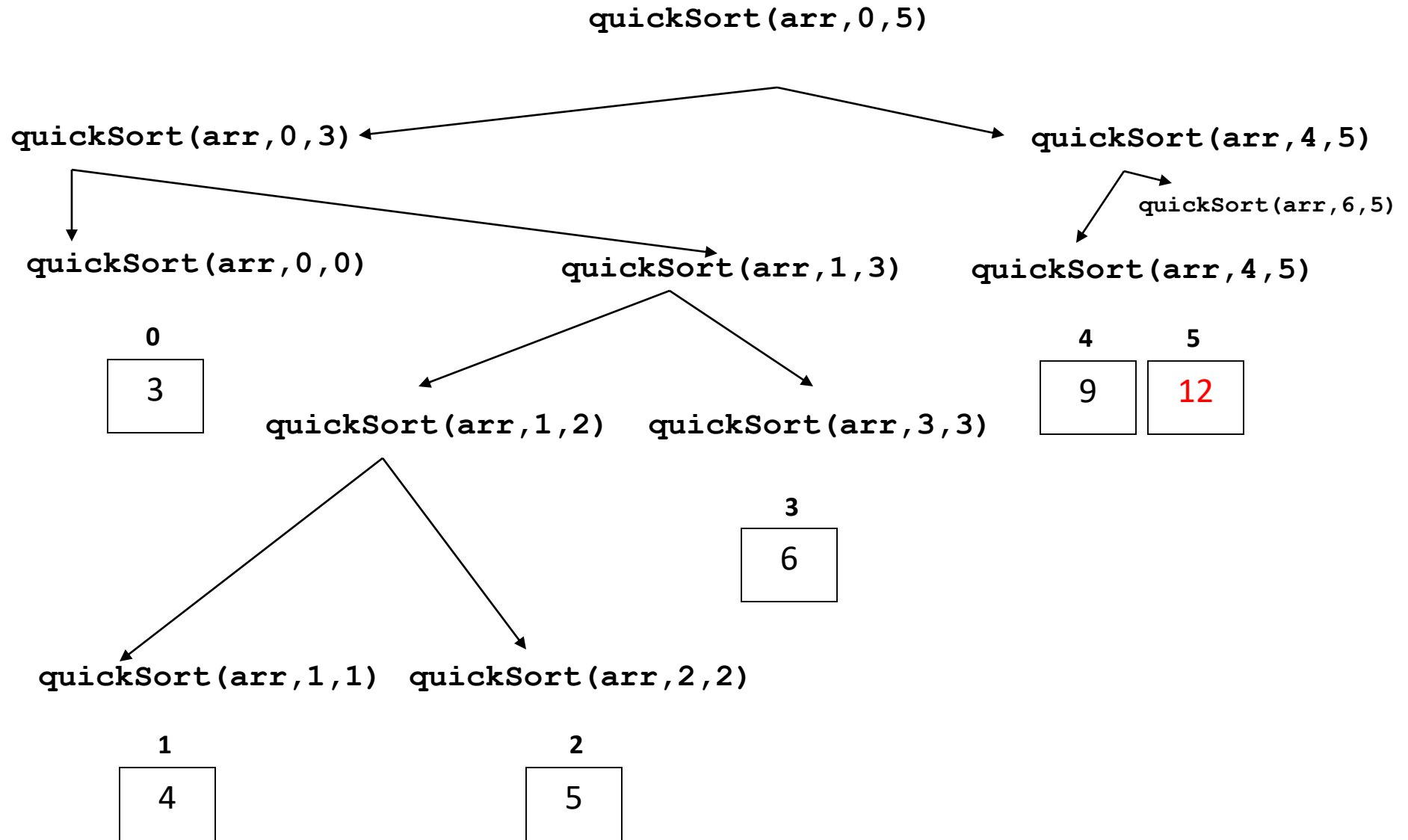
3
6

quickSort(arr, 1, 1)

quickSort(arr, 2, 2)

1
4

2
5



quickSort(arr, 0, 5)

quickSort(arr, 0, 3)

quickSort(arr, 4, 5)

quickSort(arr, 0, 0)

quickSort(arr, 1, 3)

quickSort(arr, 6, 5)

quickSort(arr, 4, 5)
partition(arr, 4, 5)

0
3

quickSort(arr, 1, 2)

quickSort(arr, 3, 3)

4
9

5
12

3
6

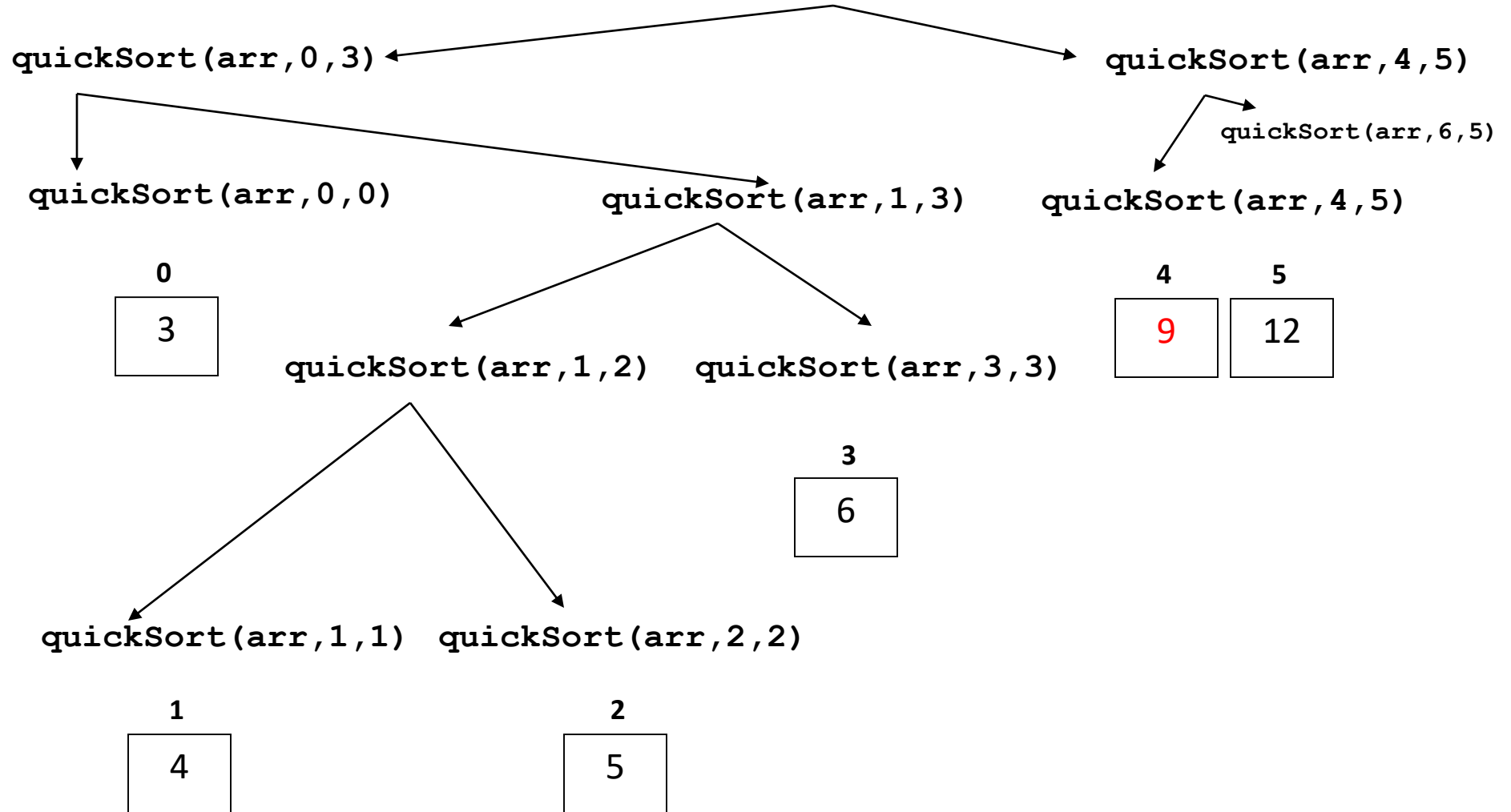
quickSort(arr, 1, 1)

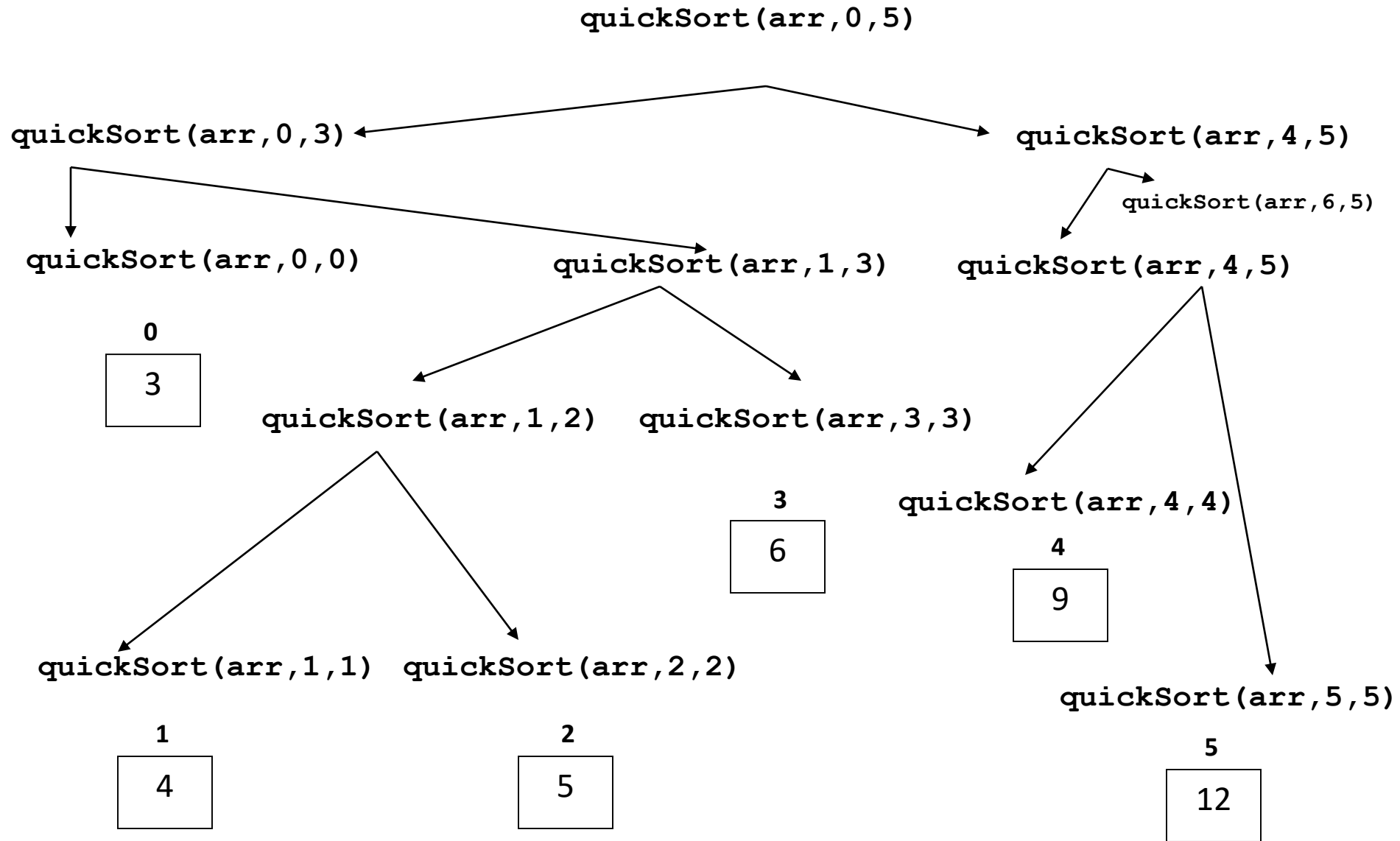
quickSort(arr, 2, 2)

1
4

2
5

`quickSort(arr, 0, 5)`





SEQUENTIAL SEARCH

- To search element position by position.
- Step through array of records, one at a time.
- Look for record with matching key.
- Search stops when
 - record with matching key is found
 - or when search has examined all records without success.

SEQUENTIAL-SEARCH(A, n, ele)

```
1. for i = 0 to n  
   if ( A [i] == ele )  
       return i;
```

- It has time complexity of $\theta(n)$.

BINARY SEARCH

- For an array of n elements, the worst case time for serial search requires n array accesses: $O(n)$.
- Consider cases where we must loop over all n records:
 - desired record appears in the last position of the array
 - desired record does not appear in the array at all
- Perhaps we can do better than $O(n)$ in the average case?
- Assume that we are given an array of records that is sorted.
- For instance:
 - an array of records with integer keys sorted from smallest to largest (e.g., ID numbers), or
 - an array of records with string keys sorted in alphabetical order (e.g., names)

BINARY SEARCH

BINARYSEARCH (A, first, last, element)

1. if(last < first)
 return (-1);
2. middle = (last + first) / 2
3. if(target == a[middle])
 return (middle)
4. else if (target < a[middle])
 // target is less than middle, so search subarray before middle
 BINARYSEARCH(A, first, middle-1, element);
5. else
 // target is greater than middle, so search subarray after middle
 BINARYSEARCH(A, middle+1, last, element);
6. End if

Binary Search

- **Binary search.** Given value and sorted array $A[]$, find index i such that $A[i] = value$, or report that no such index exists.
- Compare $A[middle]$ $((lo + hi)/2)$ element with ***value***, if ***value*** is higher than update ***lo***, otherwise update ***hi***.
- **Invariant.** Algorithm maintains $A[lo] \leq value \leq A[hi]$.
- Ex. Binary search for 33.

[illegible]

Binary Search

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↑							↑							↑
lo							mid							hi

Here, $A[\text{mid}] > \text{value} \rightarrow \text{hi} = \text{mid} - 1$

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↑						↑								
lo						hi								

Binary Search

$$\text{Mid} = (\text{lo} + \text{hi}) / 2$$

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
↑			↑			↑								
lo			mid			hi								

Here, $A[\text{mid}] < \text{value} \rightarrow \text{lo} = \text{mid} + 1$

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
				↑		↑								
				lo		hi								

Binary Search

$$\text{Mid} = (\text{lo} + \text{hi}) / 2$$

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
				↑	↑	↑								
				lo	mid	hi								

Here, $A[mid] > value \rightarrow hi = mid - 1$

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

↑
lo
hi

Binary Search

$$\text{Mid} = (\text{lo} + \text{hi}) / 2$$

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

↑
lo
hi
mid

Here, $A[mid] == \text{value} \rightarrow$ returns mid as position of value

6	13	14	25	33	43	51	53	64	72	84	93	95	96	97
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

↑
lo
hi
mid